

ĐẠI HỌC BÁCH KHOA HÀ NỘI

ĐỒ ÁN TỐT NGHIỆP

**Xây dựng nền tảng học tiếng Nhật trực tuyến qua
video kết hợp hệ thống quiz tương tác**

PHAN HOÀNG LONG

long.ph225738@sis.hust.edu.vn

Chương trình đào tạo: Việt-Nhật (IT-E6)

Giảng viên hướng dẫn: TS. Trịnh Anh Phúc

Chữ kí GVHD

Khoa: Khoa học máy tính

Trường: Công nghệ Thông tin và Truyền thông

HÀ NỘI, 06/2026

LỜI CẢM ƠN

Lời đầu tiên, em xin gửi lời cảm ơn sâu sắc tới thầy Trịnh Anh Phúc, giảng viên hướng dẫn đồ án tốt nghiệp. Thầy đã luôn theo sát, gợi ý và định hướng cho em trong suốt quá trình xây dựng và hoàn thiện hệ thống NihongoFlow, từ khâu phân tích yêu cầu, lựa chọn công nghệ, cho đến thiết kế và đánh giá hệ thống. Nhờ những góp ý quý báu của thầy, em đã có thể nhìn nhận vấn đề chặt chẽ hơn và hoàn thành đồ án một cách thuận lợi.

Em cũng xin gửi lời cảm ơn chân thành tới các thầy cô Trường Công nghệ thông tin và Truyền thông, Đại học Bách khoa Hà Nội. Với sự tận tâm và nhiệt huyết, các thầy cô đã truyền dạy cho em những nền tảng kiến thức vững vàng trong suốt quá trình học tập, là cơ sở quan trọng để em có thể thực hiện đồ án này.

Em xin gửi lời cảm ơn tới gia đình và người thân, những người luôn quan tâm, động viên và là chỗ dựa tinh thần để em vượt qua những khó khăn trong quá trình thực hiện đồ án.

Lời cuối, em xin gửi lời cảm ơn tới những người bạn đã đồng hành, hỗ trợ và góp ý cho em trong suốt khoảng thời gian làm đồ án. Do thời gian và kinh nghiệm thực tế còn hạn chế, đồ án khó tránh khỏi những thiếu sót, em rất mong nhận được sự góp ý từ thầy cô và hội đồng để hoàn thiện hơn.

TÓM TẮT NỘI DUNG ĐỒ ÁN

Nhu cầu học tiếng Nhật tại Việt Nam tăng trưởng mạnh trong nhiều năm qua, song việc học ngôn ngữ này đòi hỏi người học tiếp xúc thường xuyên với nội dung nghe nhìn thực tế và phải chủ động kiểm tra lại kiến thức ngay sau khi tiếp thu. Các nền tảng phổ biến hiện nay như Duolingo, WaniKani hay các kho video trên YouTube đều chưa giải quyết trọn vẹn bài toán này: hoặc thiếu nội dung video có cấu trúc theo lộ trình, hoặc không gắn cơ chế kiểm tra kiến thức ngay sau mỗi bài học, hoặc chưa hỗ trợ đa dạng dạng câu hỏi phù hợp đặc thù tiếng Nhật và chưa được bản địa hóa cho người học Việt Nam.

Để giải quyết vấn đề trên, em lựa chọn xây dựng NihongoFlow — một nền tảng học tiếng Nhật trực tuyến kết hợp video bài giảng với hệ thống quiz tương tác ngay sau mỗi video, theo kiến trúc web phân tách frontend và backend giao tiếp qua RESTful API. Hướng tiếp cận này cho phép em phát triển độc lập hai thành phần, đồng thời dễ mở rộng và bảo trì cho một đồ án do một sinh viên triển khai toàn bộ.

Em xây dựng backend bằng Spring Boot và cơ sở dữ liệu MySQL để quản lý toàn bộ nghiệp vụ về khóa học, bài học, quiz, tiến độ học tập và xác thực người dùng bằng JWT; phía frontend, em sử dụng React kết hợp TanStack Query và Zustand để quản lý dữ liệu và trạng thái giao diện. Nội dung học được tổ chức theo cấu trúc Cấp độ – Khóa học – Bài học – Quiz, cho phép học viên đăng ký khóa học (miễn phí hoặc trả phí qua VNPay), xem video bài giảng có theo dõi tiến độ, và làm bài quiz gồm 10 câu chia thành ba nhóm dạng câu hỏi (từ vựng, nội dung, trình tự) ngay sau khi xem hết video. Kết quả học tập, lịch sử làm bài và chuỗi ngày học liên tục được lưu lại để học viên theo dõi quá trình học của mình; phía quản trị viên có giao diện riêng để quản lý khóa học, bài học và ngân hàng câu hỏi.

Đóng góp chính của đồ án là một hệ thống web hoàn chỉnh, đã được triển khai và kiểm thử, trong đó hệ thống quiz được em thiết kế tổng quát hóa theo kiểu câu hỏi (*question type*), cho phép bổ sung các dạng câu hỏi mới trong tương lai mà không cần thay đổi cấu trúc cơ sở dữ liệu, cùng với cơ chế phân quyền tách biệt rõ ràng giữa học viên và quản trị viên, và cơ chế xác thực JWT có xoay vòng refresh token đảm bảo an toàn cho người dùng.

Sinh viên thực hiện
(Ký và ghi rõ họ tên)

ABSTRACT

The demand for learning Japanese in Vietnam has grown steadily for many years, yet learning this language requires frequent exposure to authentic audio-visual content and active recall of newly acquired knowledge immediately after exposure. Popular platforms such as Duolingo, WaniKani, and YouTube video channels do not fully address this problem: they either lack structured, level-based video content, lack a knowledge-check mechanism immediately following each lesson, or do not support the diverse question types required for Japanese, and are not localized for Vietnamese learners.

To address this problem, I built NihongoFlow, an online Japanese learning platform that combines lecture videos with an interactive quiz immediately following each video, using a web architecture that separates the frontend and backend and communicates through a RESTful API. This approach allows the two components to be developed independently while remaining easy to extend and maintain for a project built entirely by a single student developer.

The system's backend is built with Spring Boot and a MySQL database that manages all business logic for courses, lessons, quizzes, learning progress, and JWT-based user authentication; the frontend uses React together with TanStack Query and Zustand for data fetching and UI state management. Content is organized into a Level – Course – Lesson – Quiz hierarchy, allowing students to enroll in courses (free or paid via VNPay), watch lecture videos with progress tracking, and complete a 10-question quiz divided into three question-type groups (vocabulary, content, and sequence) immediately after finishing the video for the first time. Learning results, attempt history, and daily learning streaks are stored so students can track their own progress; administrators have a separate interface to manage courses, lessons, and the question bank.

The thesis's main contribution is a complete, deployed, and tested web system in which the quiz subsystem is generalized around a question-type model, allowing new question types to be added in the future without changing the database schema, together with a clear separation of access control between students and administrators and a JWT authentication scheme with refresh-token rotation for user security.

MỤC LỤC

CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI.....	1
1.1 Đặt vấn đề.....	1
1.2 Mục tiêu và phạm vi đề tài.....	2
1.3 Định hướng giải pháp.....	3
1.4 Bố cục đồ án	3
CHƯƠNG 2. KHẢO SÁT VÀ PHÂN TÍCH YÊU CẦU.....	5
2.1 Khảo sát hiện trạng	5
2.2 Tổng quan chức năng	6
2.2.1 Biểu đồ use case tổng quát	6
2.2.2 Biểu đồ use case phân rã.....	8
2.2.3 Quy trình nghiệp vụ	9
2.3 Đặc tả chức năng	11
2.3.1 Đặc tả use case “Đăng nhập”.....	12
2.3.2 Đặc tả use case “Đăng ký khóa học”	12
2.3.3 Đặc tả use case “Xem video bài học”	14
2.3.4 Đặc tả use case “Làm bài quiz”	15
2.3.5 Đặc tả use case “Quản lý câu hỏi quiz” (Quản trị viên).....	15
2.4 Yêu cầu phi chức năng	17
CHƯƠNG 3. NỀN TẢNG LÝ THUYẾT VÀ CÔNG NGHỆ SỬ DỤNG.....	18
3.1 Kiến trúc RESTful API	18
3.2 Backend: Spring Boot, Spring Security và Spring Data JPA	18
3.3 Cơ sở dữ liệu: MySQL và Flyway.....	19
3.4 Frontend: React, TypeScript, TanStack Query và Zustand.....	19
3.5 Giao diện: Tailwind CSS và shadcn/ui	20

3.6 Phát video: YouTube IFrame API.....	20
3.7 Thanh toán trực tuyến: VNPay	21
CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG	22
4.1 Thiết kế kiến trúc.....	22
4.1.1 Lựa chọn kiến trúc phần mềm	22
4.1.2 Thiết kế tổng quan.....	23
4.1.3 Thiết kế chi tiết gói	25
4.2 Thiết kế chi tiết.....	26
4.2.1 Thiết kế giao diện	26
4.2.2 Thiết kế lớp	27
4.2.3 Thiết kế cơ sở dữ liệu	29
4.3 Xây dựng ứng dụng.....	30
4.3.1 Thư viện và công cụ sử dụng.....	30
4.3.2 Kết quả đạt được	31
4.3.3 Minh họa các chức năng chính	32
4.4 Kiểm thử.....	37
4.5 Triển khai	39
CHƯƠNG 5. CÁC GIẢI PHÁP VÀ ĐÓNG GÓP NỔI BẬT.....	40
5.1 Thiết kế hệ thống quiz mở rộng theo kiểu câu hỏi	40
5.2 Phân quyền tách biệt giữa học viên và quản trị viên (Hướng A).....	41
5.3 Cơ chế xác thực JWT với refresh token xoay vòng	42
5.4 Tối ưu truy vấn N+1 trong thống kê khóa học theo cấp độ	42
5.5 Trình phát video YouTube tùy biến với luồng tự động chuyển sang quiz.....	43
5.6 Tích hợp thanh toán VNPay với cơ chế đăng ký khóa học tự động	44

CHƯƠNG 6. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	46
6.1 Kết luận.....	46
6.1.1 So sánh với các nền tảng tương tự.....	46
6.1.2 Kết quả đạt được	46
6.1.3 Hạn chế còn tồn tại.....	47
6.1.4 Bài học kinh nghiệm	47
6.2 Hướng phát triển.....	48
TÀI LIỆU THAM KHẢO.....	49
PHỤ LỤC.....	51
A. ĐẶC TẢ USE CASE BỔ SUNG.....	51
A.1 Đặc tả use case “Xem tổng quan tiến độ và streak học tập”	51
A.2 Đặc tả use case “Ấn/hiện khóa học” (Quản trị viên)	52

DANH MỤC HÌNH VẼ

Hình 2.1	Biểu đồ use case tổng quát hệ thống NihongoFlow	7
Hình 2.2	Biểu đồ use case phân rã nhóm chức năng học tập	8
Hình 2.3	Biểu đồ use case phân rã nhóm chức năng thanh toán	9
Hình 2.4	Biểu đồ use case phân rã nhóm chức năng quản trị	9
Hình 2.5	Biểu đồ hoạt động quy trình hoàn thành một bài học	11
Hình 4.1	Biểu đồ gói tổng quan hệ thống NihongoFlow	24
Hình 4.2	Biểu đồ lớp nhóm Quiz	25
Hình 4.3	Biểu đồ lớp nhóm Enrollment và Payment	26
Hình 4.4	Biểu đồ trình tự: Nộp bài quiz	28
Hình 4.5	Biểu đồ trình tự: Đăng ký khóa học có phí qua VNPAY	28
Hình 4.6	Biểu đồ thực thể liên kết (ERD) hệ thống NihongoFlow	29
Hình 4.7	Trang chủ – giới thiệu các cấp độ N5–N1	33
Hình 4.8	Trang danh sách khóa học, lọc theo cấp độ	33
Hình 4.9	Trang chi tiết khóa học – danh sách bài học và nút đăng ký/mua khóa học	34
Hình 4.10	Trang xem video bài học với YouTube custom player	34
Hình 4.11	Trang làm bài quiz theo wizard 3 bước, chia theo 3 nhóm câu hỏi	35
Hình 4.12	Trang Dashboard – tiến độ học tập, streak và lịch sử làm quiz	35
Hình 4.13	Trang quản trị – danh sách và quản lý khóa học	36
Hình 4.14	Trang quản trị – chỉnh sửa câu hỏi quiz	36
Hình 4.15	Trang kết quả thanh toán VNPAY	37

DANH MỤC BẢNG BIỂU

Bảng 2.1	So sánh NihongoFlow với các nền tảng học tiếng Nhật hiện có	5
Bảng 4.1	Danh sách công cụ và thư viện sử dụng	31
Bảng 4.2	Thống kê quy mô mã nguồn theo tầng	32
Bảng 4.3	Trường hợp kiểm thử chức năng nộp bài quiz	38
Bảng 4.4	Trường hợp kiểm thử chức năng đăng ký khóa học và thanh toán VNPAY	38

DANH MỤC THUẬT NGỮ VÀ TỪ VIẾT TẮT

Thuật ngữ	Ý nghĩa
API	Giao diện lập trình ứng dụng (Application Programming Interface)
CORS	Cơ chế cho phép chia sẻ tài nguyên giữa các nguồn gốc khác nhau (Cross-Origin Resource Sharing)
CRUD	Bốn thao tác cơ bản trên dữ liệu: tạo, đọc, cập nhật, xóa (Create, Read, Update, Delete)
DTO	Đối tượng truyền dữ liệu giữa các tầng của ứng dụng (Data Transfer Object)
ERD	Biểu đồ thực thể liên kết, mô tả cấu trúc cơ sở dữ liệu (Entity-Relationship Diagram)
HMAC	Mã xác thực thông điệp dựa trên hàm băm, dùng để xác thực dữ liệu giao dịch (Hash-based Message Authentication Code)
HTML	Ngôn ngữ đánh dấu siêu văn bản (HyperText Markup Language)
JLPT	Kỳ thi năng lực tiếng Nhật (Japanese-Language Proficiency Test)
JPA	Đặc tả ánh xạ đối tượng - quan hệ trong Java (Java Persistence API)
JSON	Định dạng trao đổi dữ liệu dạng văn bản (JavaScript Object Notation)
JWT	Token xác thực dạng JSON dùng để trao đổi thông tin định danh người dùng (JSON Web Token)
MVC	Mô hình kiến trúc phần mềm tách biệt dữ liệu, giao diện và xử lý nghiệp vụ (Model - View - Controller)

Thuật ngữ	Ý nghĩa
ORM	Kỹ thuật ánh xạ đối tượng - quan hệ (Object-Relational Mapping)
RBAC	Cơ chế phân quyền truy cập dựa trên vai trò người dùng (Role-Based Access Control)
REST	Kiến trúc giao tiếp dựa trên giao thức HTTP (Representational State Transfer)
SPA	Ứng dụng web tải một trang duy nhất và cập nhật nội dung động (Single Page Application)
UI	Giao diện người dùng (User Interface)
UX	Trải nghiệm người dùng (User Experience)

CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI

Chương này giới thiệu tổng quan về đề tài, bao gồm bối cảnh hình thành vấn đề, mục tiêu và phạm vi nghiên cứu, định hướng giải pháp em lựa chọn, và bố cục của báo cáo. Các nội dung được trình bày lần lượt trong các mục từ 1.1 đến 1.4.

1.1 Đặt vấn đề

Trong bối cảnh hội nhập kinh tế và giao lưu văn hóa ngày càng sâu rộng giữa Việt Nam và Nhật Bản, nhu cầu học tiếng Nhật tại Việt Nam tăng trưởng mạnh và ổn định trong nhiều năm qua. Theo khảo sát của Quỹ Giao lưu Quốc tế Nhật Bản năm 2021, Việt Nam có hơn 174.000 người học tiếng Nhật, xếp thứ 8 trên toàn thế giới [1]. Nhu cầu này không chỉ đến từ sinh viên và người đi làm có định hướng xuất khẩu lao động hay du học, mà còn từ lượng lớn lao động đang làm việc tại hơn 4.500 doanh nghiệp có vốn đầu tư Nhật Bản tại Việt Nam [2]. Quy mô người học lớn như vậy đặt ra yêu cầu cấp thiết về các công cụ và nền tảng học tập hiệu quả, linh hoạt về thời gian và địa điểm.

Tuy nhiên, qua tìm hiểu em nhận thấy việc học tiếng Nhật đặt ra những thách thức đặc thù so với nhiều ngôn ngữ khác. Người học phải đồng thời tiếp thu ba hệ thống chữ viết (Hiragana, Katakana và Kanji), nắm vững ngữ pháp có cấu trúc khác biệt hoàn toàn so với tiếng Việt, và rèn luyện khả năng nghe – nhận diện ngữ điệu, tốc độ nói và ngữ cảnh hội thoại tự nhiên. Những yêu cầu này đòi hỏi người học phải tiếp xúc thường xuyên với ngôn ngữ trong ngữ cảnh thực tế, không thể chỉ dựa vào việc ghi nhớ lý thuyết.

Học qua video là một hình thức đáp ứng tốt nhu cầu tiếp xúc ngôn ngữ trong ngữ cảnh thực tế, đồng thời phù hợp với lịch học linh hoạt của phần lớn người học. Dù vậy, em cho rằng học qua video đơn thuần tồn tại một bất cập nghiêm trọng: người học tiếp nhận nội dung theo chiều thụ động mà không có cơ chế buộc phải xử lý và củng cố kiến thức ngay sau đó. Các nghiên cứu về khoa học nhận thức cho thấy kiến thức chỉ được chuyển hóa vào bộ nhớ dài hạn một cách hiệu quả khi người học chủ động truy xuất thông tin ngay sau khi tiếp thu – một nguyên lý được gọi là *active recall* [3]. Khi không có bước kiểm tra này, người học có xu hướng đánh giá sai mức độ hiểu bài của bản thân, dẫn đến tình trạng quên kiến thức nhanh và thiếu cơ sở để xác định phần nội dung cần ôn luyện thêm.

Từ những phân tích trên, em xác định bài toán đặt ra là xây dựng một nền tảng kết hợp học qua video với hệ thống kiểm tra kiến thức có cấu trúc ngay sau mỗi video, trong đó hệ thống kiểm tra phải đủ linh hoạt để hỗ trợ các dạng câu hỏi đặc

thù của tiếng Nhật như nhận diện từ vựng, kiểm tra hiểu nội dung video, hay sắp xếp trình tự sự kiện. Nếu được giải quyết tốt, bài toán này mang lại lợi ích trực tiếp cho người học tiếng Nhật ở mọi trình độ – từ người mới bắt đầu đến người ôn thi JLPT – đồng thời cung cấp cho người xây dựng nội dung một công cụ quản lý và phân phối tài nguyên học tập có kiểm soát chất lượng. Về mặt rộng hơn, mô hình kết hợp video và quiz kiến thức theo cấu trúc mà em đề xuất có thể được áp dụng cho việc học bất kỳ ngôn ngữ nào khác, hoặc mở rộng sang các lĩnh vực đào tạo trực tuyến có yêu cầu đánh giá kiến thức sau từng đơn vị nội dung.

1.2 Mục tiêu và phạm vi đề tài

Trước khi bắt tay xây dựng hệ thống, em đã tìm hiểu một số nền tảng học tiếng Nhật trực tuyến đã được phát triển và triển khai rộng rãi. Duolingo cung cấp các bài học ngắn dưới dạng trò chơi hóa, phù hợp để xây dựng thói quen học nhưng thiếu cấu trúc theo lộ trình học bài bản. WaniKani tập trung vào việc ghi nhớ Kanji và từ vựng thông qua phương pháp lặp lại ngắt quãng, nhưng không tích hợp nội dung nghe nhìn. Các nền tảng chia sẻ video như YouTube có kho nội dung tiếng Nhật phong phú, song hoàn toàn không có hệ thống theo dõi tiến độ hay kiểm tra kiến thức sau mỗi bài học. Một số trang học trực tuyến như JapanesePod101 có kết hợp video và bài tập, tuy nhiên nội dung chủ yếu được thiết kế cho người dùng quốc tế, chưa được bản địa hóa cho người học Việt Nam.

Qua phân tích các nền tảng trên, em nhận thấy ba hạn chế chính: (i) thiếu sự kết hợp có cấu trúc giữa nội dung video và hệ thống luyện tập gắn liền ngay sau mỗi video; (ii) hệ thống bài kiểm tra còn đơn điệu, chưa hỗ trợ đa dạng dạng câu hỏi phù hợp với đặc thù tiếng Nhật; và (iii) chưa có nền tảng nào được thiết kế riêng cho người học Việt Nam với lộ trình theo chuẩn năng lực JLPT.

Trên cơ sở đó, em hướng tới xây dựng một nền tảng học tiếng Nhật trực tuyến kết hợp video và hệ thống quiz tương tác, với các chức năng chính gồm: (i) quản lý khóa học và bài học theo cấp độ từ sơ cấp đến N1; (ii) phát video bài học có theo dõi tiến độ xem; (iii) hệ thống quiz sau mỗi video hỗ trợ nhiều dạng câu hỏi và có khả năng mở rộng; (iv) theo dõi kết quả học tập, chuỗi ngày học liên tục (streak) và lịch sử làm bài của người dùng; và (v) cơ chế đăng ký khóa học, bao gồm cả khóa học miễn phí và khóa học trả phí thông qua cổng thanh toán trực tuyến.

Phạm vi đề án mà em đặt ra giới hạn ở việc xây dựng hệ thống cho hai nhóm người dùng chính là quản trị viên và học viên, trên nền tảng web. Các chức năng ngoài phạm vi đề án bao gồm học trực tiếp qua livestream và hệ thống gợi ý lộ trình học dựa trên trí tuệ nhân tạo – đây là những hướng mở rộng em sẽ đề cập lại ở Chương 6.

1.3 Định hướng giải pháp

Để giải quyết các hạn chế đã xác định, em áp dụng kiến trúc ứng dụng web phân tách frontend và backend theo mô hình RESTful API. Theo mô hình này, backend đảm nhận xử lý nghiệp vụ và quản lý dữ liệu, trong khi frontend chịu trách nhiệm hiển thị giao diện và tương tác với người dùng; hai thành phần giao tiếp với nhau thông qua các API chuẩn HTTP. Em lựa chọn kiến trúc này vì nó cho phép phát triển song song hai thành phần, dễ bảo trì và mở rộng trong tương lai – điều quan trọng với một đồ án do một sinh viên phát triển toàn bộ cả hai phía.

Phía backend, em xây dựng bằng Spring Boot – một framework Java trưởng thành với hệ sinh thái phong phú, hỗ trợ tốt cho việc xây dựng RESTful API và tích hợp bảo mật; em sử dụng JWT kết hợp cơ chế xoay vòng refresh token để xác thực người dùng và hạn chế rủi ro lộ token. Dữ liệu được em lưu trữ trên MySQL với thiết kế schema linh hoạt, trong đó hệ thống quiz được tổ chức theo dạng tổng quát hóa kiểu câu hỏi, cho phép bổ sung dạng câu hỏi mới mà không cần thay đổi cấu trúc cơ sở dữ liệu. Phía frontend, em sử dụng React – một thư viện JavaScript phổ biến với khả năng xây dựng giao diện theo dạng component tái sử dụng, kết hợp với TanStack Query để quản lý dữ liệu lấy từ API và Zustand để quản lý trạng thái xác thực phía client, phù hợp với yêu cầu quản lý nhiều loại câu hỏi có giao diện khác nhau.

Đóng góp chính mà em hướng tới là một hệ thống web hoàn chỉnh cho phép học tiếng Nhật qua video gắn kết với quiz đánh giá kiến thức, trong đó hệ thống quiz được em thiết kế mở rộng để hỗ trợ nhiều dạng câu hỏi khác nhau mà không phá vỡ cấu trúc hiện tại.

1.4 Bố cục đồ án

Phần còn lại của báo cáo đồ án tốt nghiệp này được em tổ chức như sau.

Chương 2 trình bày kết quả khảo sát và phân tích yêu cầu của hệ thống, bao gồm các yêu cầu chức năng và phi chức năng em xác định được, từ đó làm cơ sở cho các quyết định thiết kế ở các chương tiếp theo.

Chương 3 giới thiệu nền tảng lý thuyết và các công nghệ em sử dụng trong đề tài, bao gồm kiến trúc RESTful API, framework Spring Boot, thư viện React và hệ quản trị cơ sở dữ liệu MySQL, cùng với lý do em lựa chọn từng công nghệ.

Chương 4 trình bày toàn bộ quá trình em phân tích thiết kế, triển khai và đánh giá hệ thống, bao gồm thiết kế kiến trúc, thiết kế cơ sở dữ liệu, thiết kế giao diện và lớp, cùng kết quả xây dựng, kiểm thử và triển khai.

Chương 5 trình bày các giải pháp và đóng góp em tâm đắc nhất trong quá trình

thực hiện đồ án, tập trung vào cơ chế thiết kế hệ thống quiz linh hoạt, phân quyền hệ thống và một số bài toán kỹ thuật khác đã giải quyết.

Chương 6 tổng kết các kết quả đạt được của đồ án, đánh giá những hạn chế còn tồn tại và đề xuất các hướng phát triển trong tương lai.

CHƯƠNG 2. KHẢO SÁT VÀ PHÂN TÍCH YÊU CẦU

Chương này trình bày kết quả khảo sát hiện trạng các nền tảng học tiếng Nhật trực tuyến hiện có, từ đó em xác định các chức năng cần xây dựng cho hệ thống NihongoFlow. Mục 2.1 phân tích và so sánh các nền tảng tương tự. Mục 2.2 trình bày tổng quan chức năng thông qua biểu đồ use case và quy trình nghiệp vụ chính. Mục 2.3 đặc tả chi tiết các use case quan trọng nhất. Mục 2.4 trình bày các yêu cầu phi chức năng của hệ thống.

2.1 Khảo sát hiện trạng

Như đã trình bày ở Chương 1, em chọn khảo sát ba nhóm nền tảng học tiếng Nhật trực tuyến phổ biến hiện nay là Duolingo, WaniKani và các kho video tiếng Nhật trên YouTube, kết hợp thêm một nền tảng song ngữ tiêu biểu là JapanesePod101. Bảng 2.1 so sánh các nền tảng này theo bốn tiêu chí em cho là liên quan trực tiếp tới bài toán đặt ra: (i) nội dung video theo lộ trình, (ii) hệ thống kiểm tra kiến thức ngay sau bài học, (iii) đa dạng dạng câu hỏi, và (iv) mức độ bản địa hóa cho người học Việt Nam.

Nền tảng	Video theo lộ trình	Kiểm tra sau bài học	Đa dạng dạng câu hỏi	Bản địa hóa cho người Việt
Duolingo	Không có	Có, dạng game hóa	Trung bình (chọn đáp án, ghép từ)	Có
WaniKani	Không có	Có, dạng spaced repetition	Thấp (nhập Kanji/đọc)	Không
YouTube	Phong phú nhưng rời rạc	Không có	Không có	Một phần (phụ đề)
JapanesePod101	Có	Có, gắn với bài học	Trung bình	Không
NihongoFlow (đề xuất)	Có, theo cấp độ N5–N1	Có, ngay sau mỗi video	Cao (mở rộng theo <i>question type</i>)	Có

Bảng 2.1: So sánh NihongoFlow với các nền tảng học tiếng Nhật hiện có

Qua khảo sát, em nhận thấy các nền tảng hiện có thường chỉ giải quyết tốt một trong hai khía cạnh: hoặc cung cấp nội dung video chất lượng (YouTube) nhưng không có cơ chế kiểm tra, hoặc có cơ chế luyện tập tốt (Duolingo, WaniKani) nhưng không gắn với nội dung video theo lộ trình. Không nền tảng nào kết hợp cả hai yếu tố trong một quy trình học khép kín *xem video – làm quiz – theo dõi tiến độ*, đồng thời phục vụ riêng cho người học Việt Nam theo các cấp độ JLPT (N5–N1).

Từ kết quả khảo sát trên, em xác định hệ thống NihongoFlow cần xây dựng tối thiểu các nhóm chức năng sau:

- Quản lý nội dung học theo cấu trúc phân cấp Cấp độ – Khóa học – Bài học – Quiz;
- Phát video bài giảng có theo dõi tiến độ xem, bắt buộc xem hết video trước khi mở quiz lần đầu;
- Hệ thống quiz 10 câu hỏi/bài học, chia thành nhiều nhóm dạng câu hỏi (từ

vựng, nội dung, trình tự), có khả năng mở rộng thêm dạng câu hỏi mới;

- Theo dõi kết quả học tập: lịch sử làm bài, xem lại đáp án đúng/sai, chuỗi ngày học liên tục;
- Cơ chế đăng ký khóa học, hỗ trợ cả khóa học miễn phí và khóa học trả phí qua cổng thanh toán VNPay;
- Giao diện quản trị riêng biệt cho phép quản trị viên quản lý khóa học, bài học và ngân hàng câu hỏi.

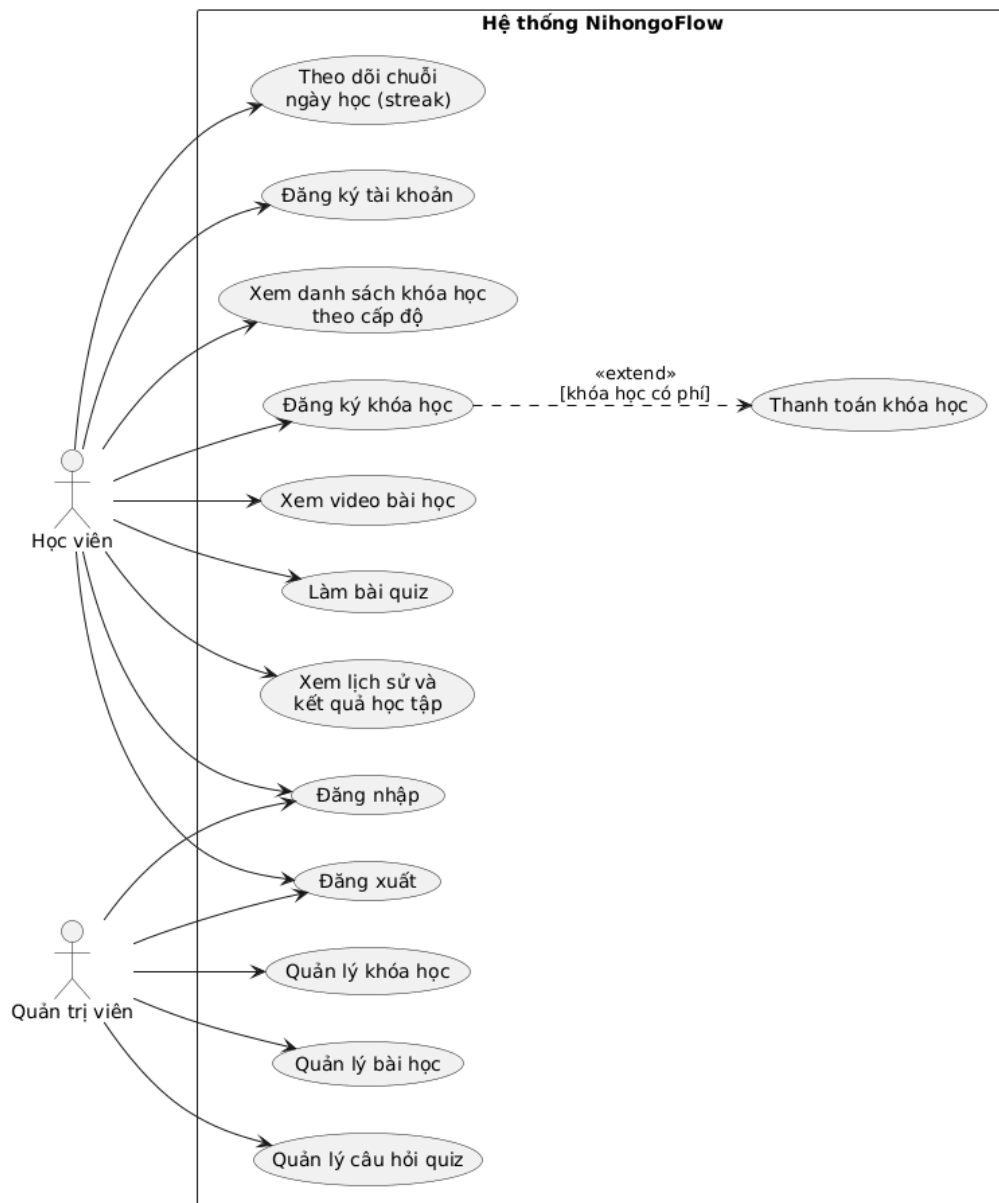
2.2 Tổng quan chức năng

2.2.1 Biểu đồ use case tổng quát

Em xác định hệ thống NihongoFlow có hai tác nhân (actor) chính:

- **Học viên:** người dùng đã đăng ký tài khoản, có thể đăng ký khóa học, xem video bài giảng, làm bài quiz, theo dõi kết quả học tập và thực hiện thanh toán cho các khóa học có phí.
- **Quản trị viên:** người quản lý nội dung của hệ thống, bao gồm cấp độ, khóa học, bài học và ngân hàng câu hỏi quiz. Quản trị viên không tham gia vào luồng học tập của học viên – lý do cho quyết định phân quyền tách biệt này được em trình bày tại Mục 5.2.

Biểu đồ use case tổng quát của hệ thống được minh họa trong Hình 2.1.



Hình 2.1: Biểu đồ use case tổng quát hệ thống NihongoFlow

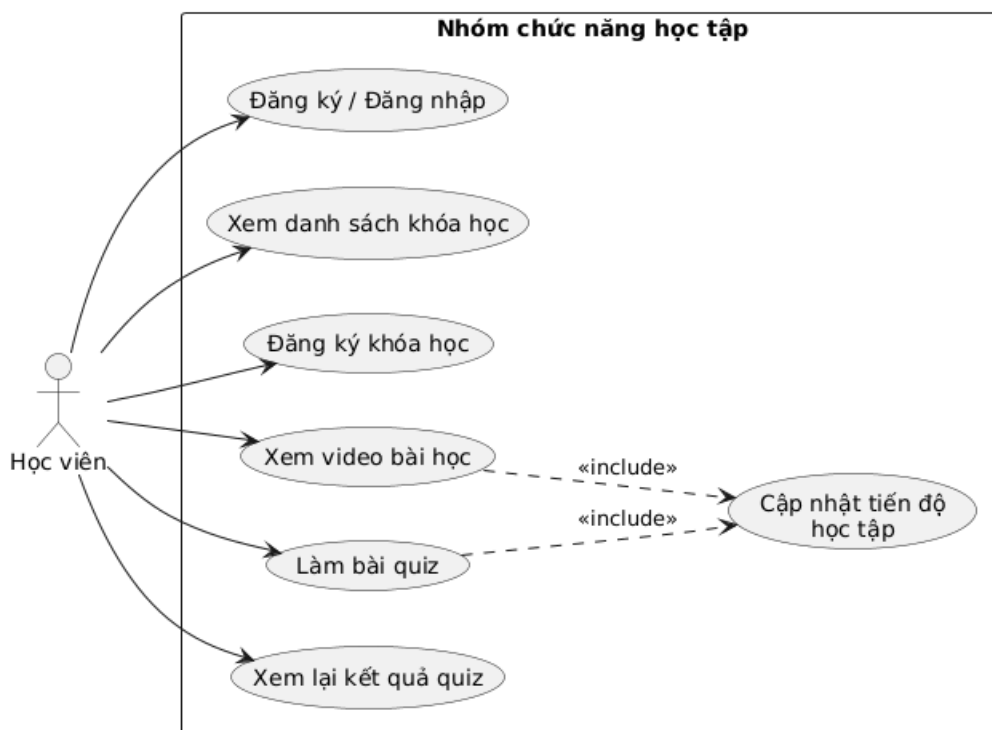
Đối với tác nhân Học viên, các use case chính bao gồm: *Đăng ký tài khoản*, *Đăng nhập*, *Đăng xuất*, *Xem danh sách khóa học theo cấp độ*, *Đăng ký khóa học*, *Thanh toán khóa học*, *Xem video bài học*, *Làm bài quiz*, *Xem lịch sử và kết quả học tập*, và *Theo dõi chuỗi ngày học*.

Đối với tác nhân Quản trị viên, các use case chính bao gồm: *Quản lý khóa học* (tạo, sửa, xóa, ẩn/hiện), *Quản lý bài học* trong từng khóa học, và *Quản lý câu hỏi quiz* của từng bài học.

2.2.2 Biểu đồ use case phân rã

a, Nhóm chức năng học tập

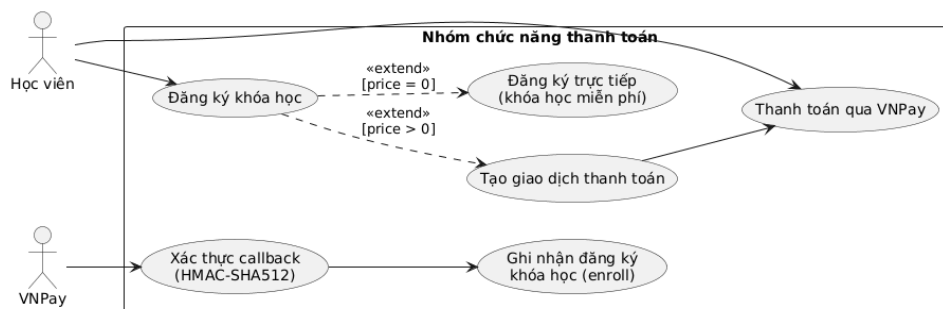
Nhóm chức năng học tập là nhóm chức năng trung tâm của hệ thống, bao phủ toàn bộ hành trình của học viên từ khi đăng ký tài khoản đến khi hoàn thành một bài học. Hình 2.2 mô tả biểu đồ use case phân rã của nhóm chức năng này, bao gồm các use case: *Đăng ký/Đăng nhập*, *Xem danh sách khóa học*, *Đăng ký khóa học*, *Xem video bài học*, *Làm bài quiz*, và *Xem lại kết quả quiz*. Trong đó, use case *Xem video bài học* và *Làm bài quiz* có quan hệ «include» với use case *Cập nhật tiến độ học tập*, vì mỗi lần học viên xem hết video hoặc nộp bài quiz, hệ thống đều cần cập nhật `UserLessonProgress` và chuỗi ngày học.



Hình 2.2: Biểu đồ use case phân rã nhóm chức năng học tập

b, Nhóm chức năng thanh toán

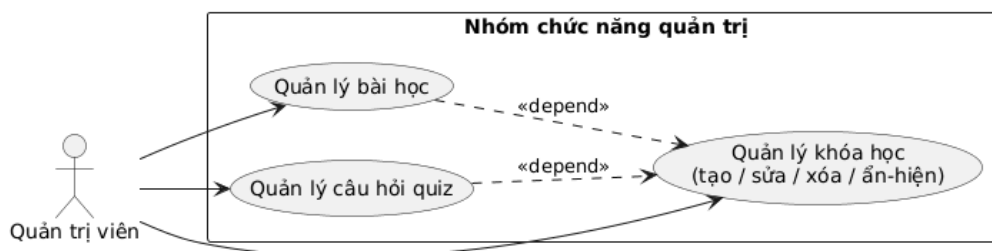
Use case *Đăng ký khóa học* ở mục trên được em phân rã tiếp theo hai nhánh tùy theo khóa học là miễn phí hay trả phí, minh họa trong Hình 2.3. Nếu khóa học miễn phí ($price = 0$), học viên được ghi nhận đăng ký (`UserCourseEnrollment`) ngay lập tức. Nếu khóa học có phí, hệ thống tạo một giao dịch thanh toán (`Payment`) và chuyển hướng học viên sang cổng thanh toán VNPay; chỉ sau khi VNPay xác nhận giao dịch thành công qua callback, hệ thống mới ghi nhận đăng ký khóa học cho học viên.



Hình 2.3: Biểu đồ use case phân rã nhóm chức năng thanh toán

c, Nhóm chức năng quản trị

Nhóm chức năng quản trị phục vụ tác nhân Quản trị viên, minh họa trong Hình 2.4, bao gồm các use case: *Quản lý khóa học* (tạo/sửa/xóa/ẩn-hiện), *Quản lý bài học* trong khóa học (tạo/sửa/xóa, tự động đọc thời lượng từ video YouTube), và *Quản lý câu hỏi quiz* (sửa nội dung câu hỏi, đáp án và đáp án đúng cho từng câu trong số 10 câu của một quiz). Các use case *Quản lý bài học* và *Quản lý câu hỏi quiz* phụ thuộc vào *Quản lý khóa học*, vì bài học và quiz luôn thuộc về một khóa học cụ thể.



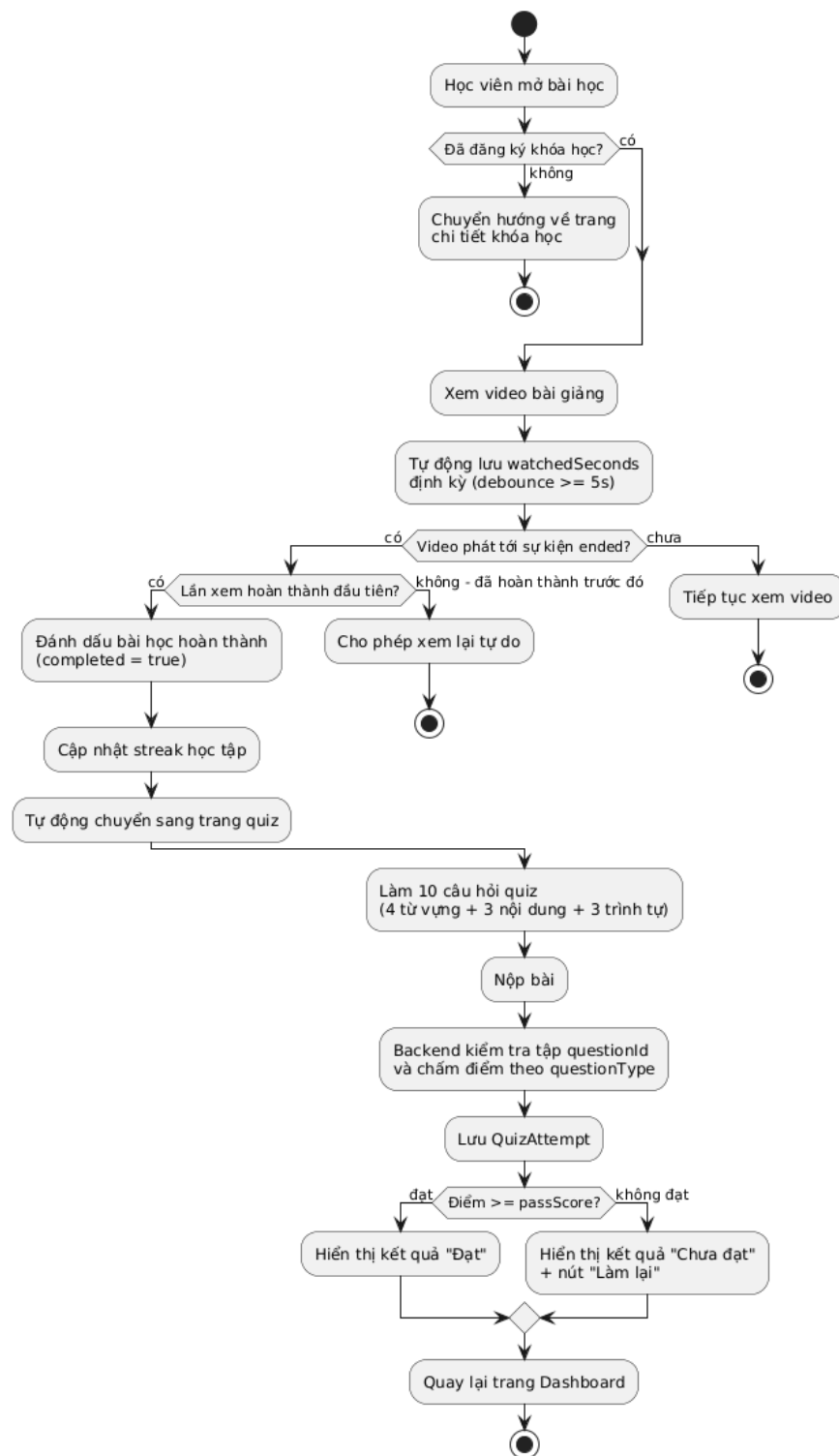
Hình 2.4: Biểu đồ use case phân rã nhóm chức năng quản trị

2.2.3 Quy trình nghiệp vụ

Quy trình nghiệp vụ quan trọng nhất của hệ thống, theo em đánh giá, là quy trình *học viên hoàn thành một bài học*, kết hợp nhiều use case: *Xem video bài học*, *Cập nhật tiến độ*, *Làm bài quiz*, và *Cập nhật streak*. Quy trình này được em mô tả sơ bộ như sau và minh họa bằng biểu đồ hoạt động trong Hình 2.5:

1. Học viên mở một bài học trong khóa học đã đăng ký. Hệ thống kiểm tra học viên đã đăng ký khóa học hay chưa; nếu chưa, chuyển hướng về trang chi tiết khóa học.
2. Học viên xem video bài giảng. Trong quá trình xem, hệ thống tự động lưu vị trí xem (watchedSeconds) định kỳ.

3. Khi video phát đến hết (sự kiện `ended`), hệ thống đánh dấu bài học là đã hoàn thành (nếu đây là lần xem đầu tiên), cập nhật streak của học viên, và tự động chuyển sang trang quiz của bài học.
4. Học viên làm 10 câu hỏi quiz, chia thành ba nhóm: từ vựng, nội dung, trình tự, sau đó nộp bài.
5. Hệ thống chấm điểm, lưu lại lượt làm bài (`QuizAttempt`) kèm đáp án đã chọn, so sánh điểm số với ngưỡng đạt (`passScore`) của quiz, và hiển thị kết quả cho học viên qua cửa sổ popup.
6. Học viên có thể quay lại trang tổng quan để xem lại lịch sử làm bài, hoặc xem lại video bài học – ở các lần xem sau, học viên không bị bắt buộc làm lại quiz.



Hình 2.5: Biểu đồ hoạt động quy trình hoàn thành một bài học

2.3 Đặc tả chức năng

Trong số các use case đã trình bày ở Mục 2.2, em lựa chọn năm use case sau để đặc tả chi tiết vì chúng đại diện cho các luồng nghiệp vụ cốt lõi và phức tạp nhất của hệ thống. Hai use case bổ sung khác được em đặc tả tại Phụ lục A do giới hạn dung lượng chương này.

2.3.1 Đặc tả use case “Đăng nhập”

Tác nhân: Học viên, Quản trị viên.

Mô tả: Người dùng đăng nhập vào hệ thống bằng email và mật khẩu để nhận access token và refresh token.

Tiền điều kiện: Người dùng đã có tài khoản trong hệ thống.

Hậu điều kiện: Người dùng được xác thực; access token được lưu ở phía client, refresh token được lưu trong cookie httpOnly; người dùng được điều hướng tới trang phù hợp với vai trò (/dashboard đối với học viên, /admin/khoa-hoc đối với quản trị viên).

Luồng sự kiện chính:

1. Người dùng nhập email và mật khẩu, nhấn nút "Đăng nhập".
2. Hệ thống gửi yêu cầu `POST/api/v1/auth/login` kèm thông tin đăng nhập.
3. Backend kiểm tra email tồn tại và mật khẩu khớp (so sánh bằng BCrypt).
4. Backend sinh access token (JWT, hết hạn sau 15 phút) chứa `userId` và `role`, sinh refresh token mới và lưu vào cơ sở dữ liệu.
5. Backend trả về access token cùng thông tin người dùng trong nội dung phản hồi, đồng thời thiết lập refresh token vào cookie httpOnly.
6. Frontend lưu access token và thông tin người dùng vào store (Zustand), điều hướng người dùng theo vai trò.

Luồng rẽ nhánh:

- *A1 – Sai email hoặc mật khẩu:* Hệ thống trả về lỗi UNAUTHORIZED, frontend hiển thị thông báo "Email hoặc mật khẩu không đúng".
- *A2 – Access token hết hạn khi đang sử dụng:* Frontend tự động gọi `POST/api/v1/auth/refresh` bằng refresh token trong cookie để lấy access token mới mà không yêu cầu người dùng đăng nhập lại.

2.3.2 Đặc tả use case “Đăng ký khóa học”

Tác nhân: Học viên.

Mô tả: Học viên đăng ký tham gia một khóa học để có thể truy cập các bài học bên trong. Nếu khóa học có phí, học viên cần thanh toán qua VNPay trước khi được ghi nhận đăng ký.

Tiền điều kiện: Học viên đã đăng nhập; khóa học đang ở trạng thái hiển thị

(hidden = false); học viên chưa đăng ký khóa học này.

Hậu điều kiện: Bản ghi `UserCourseEnrollment` được tạo cho cặp (học viên, khóa học); học viên có thể truy cập tất cả bài học trong khóa học.

Luồng sự kiện chính (khóa học miễn phí):

1. Học viên xem trang chi tiết khóa học, nhấn nút "Đăng ký học".
2. Frontend gửi yêu cầu `POST/api/v1/courses/{courseId}/enroll`.
3. Backend kiểm tra học viên chưa đăng ký, tạo bản ghi `UserCourseEnrollment` với thời điểm đăng ký hiện tại.
4. Hệ thống trả về thành công; frontend cập nhật giao diện, mở khóa toàn bộ bài học trong khóa học.

Luồng sự kiện chính (khóa học có phí – tích hợp với thanh toán VNPay):

1. Học viên xem trang chi tiết khóa học, thấy giá tiền và nhấn nút "Mua khóa học".
2. Frontend gửi yêu cầu `POST/api/v1/payments/create` kèm `courseId`.
3. Backend tạo bản ghi `Payment` ở trạng thái `PENDING` với mã giao dịch (`vnpxnRef`) duy nhất, sinh URL thanh toán VNPay (ký bằng HMAC-SHA512) và trả về cho frontend.
4. Frontend chuyển hướng trình duyệt (`window.location.href`) sang cổng thanh toán VNPay.
5. Học viên hoàn tất thanh toán trên VNPay. VNPay gọi lại (`GET/api/v1/payments/vnpay-return`) tới backend kèm chữ ký giao dịch.
6. Backend xác thực chữ ký HMAC-SHA512; nếu hợp lệ và giao dịch thành công, cập nhật `Payment.status=SUCCESS` và tạo bản ghi `UserCourseEnrollment` cho học viên.
7. Backend chuyển hướng học viên về trang `/thanh-toan/ket-qua` của frontend, hiển thị kết quả thanh toán và nút "Vào học ngay" nếu thành công.

Luồng rẽ nhánh:

- *A1 – Học viên đã đăng ký khóa học:* Backend trả về lỗi, frontend ẩn nút đăng ký và hiển thị trạng thái "Đã đăng ký".
- *A2 – Chữ ký VNPay không hợp lệ hoặc giao dịch thất bại:* `Payment.status`

= FAILED, học viên không được ghi nhận đăng ký, trang kết quả hiển thị thông báo thất bại bằng tiếng Việt.

- A3 – *VNPay gọi lại callback nhiều lần cho cùng một giao dịch*: Nếu Payment đã ở trạng thái SUCCESS từ lần gọi trước, backend trả ngay kết quả thành công mà không xử lý lại hay enroll lại, tránh ghi đè dữ liệu giao dịch gốc.

2.3.3 Đặc tả use case “Xem video bài học”

Tác nhân: Học viên.

Mô tả: Học viên xem video bài giảng (nhúng từ YouTube); hệ thống theo dõi tiến độ xem và tự động chuyển sang trang quiz khi xem hết video lần đầu.

Tiền điều kiện: Học viên đã đăng ký khóa học chứa bài học này.

Hậu điều kiện: `UserLessonProgress.watchedSeconds` được cập nhật; nếu là lần xem đầu tiên và xem hết video, `completed = true` và `streak` của học viên được cập nhật.

Luồng sự kiện chính:

1. Học viên mở trang bài học (`/khoa-hoc/:courseId/bai-hoc/:lessonId`).
2. Frontend gọi `GET/api/v1/lessons/{lessonId}` để lấy thông tin video và tiến độ đã lưu, video tự động phát tiếp từ vị trí `watchedSeconds` đã lưu trước đó.
3. Trong quá trình phát, frontend định kỳ gửi `POST/api/v1/lessons/{lessonId}/progress` để cập nhật `watchedSeconds` (chỉ gửi khi độ lệch thời gian ≥ 5 giây).
4. Khi video phát tới sự kiện ended: nếu đây là lần xem hoàn thành đầu tiên, frontend gửi cùng yêu cầu cập nhật tiến độ với cờ `completed = true`; backend đánh dấu bài học hoàn thành, cập nhật `streak`, và frontend tự động điều hướng sang trang quiz của bài học.

Luồng rẽ nhánh:

- A1 – *Bài học đã hoàn thành trước đó*: Khi video kết thúc, hệ thống không điều hướng sang quiz, cho phép học viên xem lại tự do.
- A2 – *Video chưa được cập nhật (URL trống)*: Frontend hiển thị thông báo "Video đang được cập nhật. Vui lòng quay lại sau."
- A3 – *Lỗi tải video*: Frontend hiển thị thông báo "Không thể tải video. Vui lòng thử lại sau."

2.3.4 Đặc tả use case “Làm bài quiz”

Tác nhân: Học viên.

Mô tả: Học viên trả lời 10 câu hỏi của bài học, chia thành 3 nhóm (từ vựng, nội dung, trình tự), và nộp bài để được chấm điểm.

Tiền điều kiện: Học viên đã đăng ký khóa học chứa bài học; quiz của bài học đã có đủ 10 câu hỏi.

Hậu điều kiện: Một bản ghi `QuizAttempt` mới được tạo, lưu lại điểm số, đáp án đã chọn và kết quả đạt/không đạt; lịch sử làm bài của học viên được cập nhật (không xóa các lượt làm trước).

Luồng sự kiện chính:

1. Frontend gọi `GET/api/v1/lessons/{lessonId}/quiz` để lấy danh sách 10 câu hỏi (không kèm đáp án đúng), hiển thị tuần tự theo 3 trang ứng với 3 nhóm (VOCABULARY, CONTENT, SEQUENCE).
2. Học viên chọn đáp án cho từng câu hỏi ở mỗi trang, sau đó nhấn "Tiếp theo"; ở trang cuối, học viên nhấn "Nộp bài".
3. Frontend gửi `POST/api/v1/quizzes/{quizId}/attempts` kèm bản đồ `questionId` sang đáp án đã chọn (chuỗi văn bản với câu từ vựng, số thứ tự đáp án với câu nội dung, mảng thứ tự với câu trình tự).
4. Backend kiểm tra tập `questionId` gửi lên trùng khớp đối xứng (`ids.equals(questionIds)`) với tập câu hỏi của quiz; nếu không khớp, trả về lỗi `VALIDATION_ERROR`.
5. Backend chấm điểm theo từng loại câu hỏi, tính tỉ lệ đúng, so sánh với `passScore` của quiz để xác định `passed`, lưu bản ghi `QuizAttempt`.
6. Backend trả về kết quả; frontend hiển thị popup kết quả (điểm số, đạt/không đạt, ngưỡng đạt) với nút "Về tổng quan" (và nút "Làm lại" nếu không đạt).

Luồng rẽ nhánh:

- *A1 – Học viên không trả lời đủ câu hỏi của một trang:* Frontend chặn chuyển sang trang tiếp theo (hoặc nộp bài) cho tới khi đã trả lời đầy đủ.
- *A2 – Học viên làm lại quiz:* Một bản ghi `QuizAttempt` mới được tạo, không ảnh hưởng tới các lượt làm trước (số lần làm không giới hạn).

2.3.5 Đặc tả use case “Quản lý câu hỏi quiz” (Quản trị viên)

Tác nhân: Quản trị viên.

Mô tả: Quản trị viên xem và chỉnh sửa nội dung, các phương án trả lời và đáp án đúng của từng câu hỏi trong quiz của một bài học.

Tiền điều kiện: Người dùng đã đăng nhập với vai trò ADMIN; bài học đã có quiz với 10 câu hỏi được tạo từ dữ liệu khởi tạo.

Hậu điều kiện: Nội dung câu hỏi, các phương án và/hoặc đáp án đúng được cập nhật trong cơ sở dữ liệu; thay đổi có hiệu lực ngay với học viên ở lượt làm quiz tiếp theo.

Luồng sự kiện chính:

1. Quản trị viên truy cập trang `/admin/khoa-hoc/:courseId/bai-hoc/:lessonId/quiz`.
2. Frontend gọi `GET/api/v1/admin/lessons/{lessonId}/quiz`, nhận về 10 câu hỏi (kèm đáp án đúng) chia theo 3 nhóm: VOCABULARY (4 câu), CONTENT (3 câu), SEQUENCE (3 câu).
3. Quản trị viên chọn một câu hỏi, nhấn nút sửa để mở modal chỉnh sửa nội dung câu hỏi và đáp án đúng theo đúng loại câu hỏi đó (đáp án văn bản cho câu từ vựng; 4 phương án và đáp án đúng cho câu nội dung; thứ tự đúng cho câu trình tự).
4. Quản trị viên lưu thay đổi; frontend gửi `PUT/api/v1/admin/questions/{questionId}` kèm các trường tương ứng.
5. Backend cập nhật bản ghi `Question` tương ứng – chỉ ghi đè trường đáp án đúng phù hợp với loại câu hỏi đang sửa – và trả về dữ liệu đã cập nhật; frontend cập nhật giao diện.

Luồng rẽ nhánh:

- A1 – Dữ liệu đáp án không hợp lệ (ví dụ thiếu đáp án văn bản cho câu từ vựng, hoặc thứ tự câu trình tự không đủ 4 vị trí): Backend trả về lỗi `VALIDATION_ERROR`.
- A2 – Người dùng không có vai trò ADMIN truy cập API: Backend trả về lỗi `FORBIDDEN`, frontend (`AdminRoute`) chặn truy cập trang trước khi gọi API.

Lưu ý: Quản trị viên chỉ chỉnh sửa nội dung của 10 câu hỏi đã có sẵn theo cấu trúc cố định (4 VOCABULARY + 3 CONTENT + 3 SEQUENCE), không thêm/xóa câu hỏi – lý do cho quyết định này được em trình bày tại Chương 1.

2.4 Yêu cầu phi chức năng

- **Bảo mật:** Mật khẩu người dùng được băm bằng BCrypt; xác thực bằng JWT (access token 15 phút) kết hợp refresh token lưu trong cookie `httpOnly` với cơ chế xoay vòng; phân quyền RBAC tách biệt hoàn toàn giữa vai trò STUDENT và ADMIN ở cả backend (`SecurityConfig`) và frontend (`route guard`); callback thanh toán VNPAY được xác thực bằng chữ ký HMAC-SHA512.
- **Hiệu năng:** Các truy vấn thống kê (ví dụ đếm số khóa học theo từng cấp độ) được em tối ưu thành truy vấn gộp (`GROUP BY`) thay vì truy vấn lặp theo từng bản ghi (vấn đề N+1), được em trình bày chi tiết tại Mục 5.4.
- **Khả năng mở rộng:** Cấu trúc bảng `questions` tổng quát hóa theo `questionType` và cột `options` dạng JSON cho phép bổ sung dạng câu hỏi mới (nghe, điền từ, nối cặp, v.v.) mà không cần thay đổi schema cơ sở dữ liệu.
- **Toàn vẹn dữ liệu:** Lịch sử làm bài quiz (`QuizAttempt`) không bao giờ bị xóa; xóa khóa học/bài học sẽ tự động xóa theo (`ON DELETE CASCADE`) các bài học, quiz, câu hỏi, lượt làm bài và tiến độ liên quan, đảm bảo không còn dữ liệu mồ côi; lược đồ cơ sở dữ liệu được quản lý phiên bản bằng Flyway.
- **Tính khả dụng và trải nghiệm người dùng:** Toàn bộ giao diện bằng tiếng Việt; mọi thao tác đều có phản hồi trạng thái (loading, lỗi, thành công); thiết kế desktop-first, hỗ trợ responsive cơ bản; nội dung tiếng Nhật hiển thị tối thiểu 14px với font Noto Sans JP để đảm bảo dễ đọc.
- **Khả năng triển khai:** Hệ thống chạy độc lập trên máy chủ với MySQL (qua Docker Compose), backend Spring Boot và frontend React build tĩnh, không phụ thuộc dịch vụ lưu trữ video bên thứ ba trả phí (sử dụng YouTube embed).

Chương tiếp theo trình bày nền tảng lý thuyết và các công nghệ em lựa chọn để giải quyết các yêu cầu chức năng và phi chức năng đã xác định ở trên.

CHƯƠNG 3. NỀN TẢNG LÝ THUYẾT VÀ CÔNG NGHỆ SỬ DỤNG

Chương này giới thiệu nền tảng lý thuyết và các công nghệ em sử dụng để xây dựng hệ thống NihongoFlow. Với mỗi công nghệ, em trình bày vấn đề/yêu cầu đã xác định ở Chương 2 mà công nghệ đó giải quyết, các lựa chọn thay thế em đã cân nhắc, và lý do lựa chọn cuối cùng.

3.1 Kiến trúc RESTful API

Để giải quyết yêu cầu phân tách rõ ràng giữa xử lý nghiệp vụ và hiển thị giao diện (Chương 2), em sử dụng kiến trúc RESTful API [4]. Theo kiến trúc này, backend cung cấp các tài nguyên (resource) như `courses`, `lessons`, `quizzes` thông qua các phương thức HTTP chuẩn (`GET`, `POST`, `PUT`, `PATCH`, `DELETE`); mỗi phản hồi đều ở định dạng JSON và được em bọc trong một "envelope" thống nhất gồm trường `success`, `data` (hoặc `error`) và `message` tùy chọn.

Lựa chọn thay thế em cân nhắc là GraphQL, cho phép client tự định nghĩa cấu trúc dữ liệu cần lấy. Tuy nhiên, GraphQL đòi hỏi thiết lập schema phức tạp hơn và không cần thiết với quy mô API của đồ án (khoảng 30 endpoint với cấu trúc dữ liệu tương đối ổn định). Vì vậy, em chọn RESTful API với envelope chuẩn hóa vì đơn giản, dễ kiểm thử (qua Postman/curl) và phù hợp với quy mô một sinh viên phát triển độc lập.

3.2 Backend: Spring Boot, Spring Security và Spring Data JPA

Backend của hệ thống đảm nhận toàn bộ xử lý nghiệp vụ: quản lý nội dung học, chấm điểm quiz, theo dõi tiến độ, xác thực người dùng và xử lý thanh toán. Em sử dụng Spring Boot 3 [5] – một framework Java phổ biến cho phép xây dựng ứng dụng web độc lập (embedded server) với cấu hình tối thiểu.

Spring Data JPA được em dùng làm tầng truy cập dữ liệu (lớp *repository*), ánh xạ các thực thể Java sang bảng MySQL thông qua Hibernate, giúp giảm lượng mã truy vấn thủ công cho các thao tác CRUD và các truy vấn tùy biến (ví dụ đếm số khóa học theo cấp độ – xem Mục 5.4).

Spring Security kết hợp với JWT [6] được em dùng để xác thực và phân quyền. Mỗi request tới API (trừ các endpoint công khai) phải kèm access token trong header `Authorization: Bearer <token>`; một bộ lọc (`JwtAuthenticationFilter`) giải mã token, xác định `userId` và `role`, từ đó `SecurityConfig` áp dụng chính sách phân quyền theo vai trò (RBAC) cho từng nhóm endpoint (`/api/v1/admin/**` chỉ dành cho ADMIN, `/api/v1/users/me/**` chỉ dành cho STUDENT).

Các lựa chọn thay thế em cân nhắc gồm Node.js với Express/NestJS, và Python với Django REST Framework. Cả hai đều có hệ sinh thái tốt cho xây dựng REST API, tuy nhiên em chọn Spring Boot vì: (i) hệ sinh thái Spring Security cung cấp sẵn các thành phần xác thực JWT, mã hóa mật khẩu (BCrypt) và phân quyền theo vai trò một cách tường minh, dễ kiểm soát; (ii) Spring Data JPA giúp định nghĩa schema thông qua entity Java, thuận tiện khi kết hợp với Flyway để quản lý migration; (iii) em đã có kinh nghiệm với Java từ chương trình đào tạo, giúp tiết kiệm thời gian xử lý các vấn đề kỹ thuật phát sinh.

3.3 Cơ sở dữ liệu: MySQL và Flyway

Dữ liệu của hệ thống có cấu trúc quan hệ rõ ràng: nội dung được tổ chức theo cây phân cấp Cấp độ – Khóa học – Bài học – Quiz – Câu hỏi, và có nhiều ràng buộc khóa ngoại quan trọng (ví dụ một lượt làm quiz luôn gắn với một quiz và một người dùng cụ thể). Vì vậy, em sử dụng MySQL [7] – một hệ quản trị cơ sở dữ liệu quan hệ phổ biến, có hỗ trợ tốt cho kiểu dữ liệu JSON (em dùng cho cột `options` của câu hỏi và `answers` của lượt làm bài).

Flyway được em sử dụng để quản lý lược đồ cơ sở dữ liệu thông qua tệp migration, cho phép áp dụng các thay đổi schema (tạo bảng, thêm ràng buộc ON DELETE CASCADE, v.v.) một cách có kiểm soát và tái lập được trên nhiều môi trường (máy cá nhân, máy chủ triển khai) chỉ bằng một lệnh khởi động ứng dụng, thay vì phải chạy thủ công các câu lệnh SQL.

Lựa chọn thay thế em cân nhắc là cơ sở dữ liệu NoSQL như MongoDB, phù hợp khi dữ liệu có cấu trúc linh hoạt, thay đổi thường xuyên. Tuy nhiên, dữ liệu của hệ thống (người dùng, khóa học, bài học, lượt làm quiz) có quan hệ chặt chẽ và cần đảm bảo tính toàn vẹn tham chiếu (ví dụ không thể có lượt làm quiz mà không có quiz tương ứng), nên em đánh giá cơ sở dữ liệu quan hệ phù hợp hơn. Tính linh hoạt cho các dạng câu hỏi khác nhau được em giải quyết bằng cột JSON trong MySQL thay vì chuyển hẳn sang mô hình NoSQL.

3.4 Frontend: React, TypeScript, TanStack Query và Zustand

Frontend của hệ thống là một ứng dụng SPA em xây dựng bằng React [8] với Vite làm công cụ build, và TypeScript ở chế độ *strict* để đảm bảo an toàn kiểu dữ liệu khi làm việc với nhiều loại DTO trả về từ backend.

TanStack Query được em dùng để quản lý dữ liệu lấy từ API (server state): caching, tự động làm mới, và đồng bộ trạng thái loading/error cho các thao tác như lấy danh sách khóa học, nộp bài quiz, hay các thao tác CRUD phía quản trị. Toàn bộ `useQuery/useMutation` được em tổ chức trong các custom hook theo domain (`useCourse`, `useQuiz`, `useDashboard`, v.v.), tách biệt khỏi tầng hiển thị.

Zustand được em dùng để quản lý trạng thái xác thực phía client (thông tin người dùng và access token hiện tại) – một trạng thái toàn cục, đơn giản, không cần đến các khái niệm action/reducer phức tạp như Redux.

Lựa chọn thay thế em cân nhắc gồm Vue.js và Angular cho framework giao diện, và Redux Toolkit cho quản lý state. Em chọn React vì mô hình component dựa trên hàm và hook phù hợp với việc xây dựng nhiều loại giao diện câu hỏi quiz có thể tái sử dụng; em chọn TanStack Query kết hợp Zustand thay Redux vì tách biệt rõ ràng giữa "dữ liệu từ server" (do TanStack Query quản lý, có caching và invalidation) và "trạng thái client thuần túy" (do Zustand quản lý), giảm lượng mã boilerplate so với Redux.

3.5 Giao diện: Tailwind CSS và shadcn/ui

Để đáp ứng yêu cầu về giao diện tối giản, nhất quán và dễ bảo trì, em sử dụng Tailwind CSS [9] – một thư viện utility-first CSS cho phép định nghĩa style trực tiếp qua class trong JSX mà không cần viết file CSS riêng cho từng component. Bộ thành phần giao diện cơ bản (nút bấm, input, badge, v.v.) em dùng shadcn/ui làm nền, tùy biến lại theo bảng màu tối tối giản (lấy cảm hứng từ phong cách thiết kế của Resend.com) thông qua các biến CSS dùng chung trong `globals.css`.

Lựa chọn thay thế là CSS Module hoặc styled-components cho từng component riêng lẻ. Em chọn Tailwind CSS vì giúp duy trì tính nhất quán của design system (màu sắc, khoảng cách theo lưới 8px) thông qua một bộ token dùng chung, đồng thời giảm số lượng file CSS cần quản lý trong một dự án có nhiều trang.

3.6 Phát video: YouTube IFrame API

Yêu cầu phát video bài giảng có theo dõi tiến độ và tự động chuyển sang quiz khi xem hết video (Chương 2) đòi hỏi truy cập được các sự kiện phát video (bắt đầu, tạm dừng, kết thúc) và thời lượng video. Em sử dụng YouTube IFrame Player API [10], cho phép nhúng video YouTube với bộ điều khiển tùy biến hoàn toàn (ẩn control mặc định, dùng overlay riêng cho play/pause/seek/mute/fullscreen) và lắng nghe sự kiện `onStateChange` để phát hiện khi video kết thúc (ENDED).

Các lựa chọn thay thế em đã cân nhắc và quyết định không sử dụng gồm Cloudinary (lưu trữ và streaming video HLS) và Cloudflare Stream. Cả hai đều yêu cầu chi phí lưu trữ/băng thông và cấu hình credentials bổ sung, trong khi đề án được đánh giá theo tính năng hệ thống chứ không theo hạ tầng lưu trữ video. Em chọn YouTube embed vì cho phép tận dụng kho video tiếng Nhật chất lượng cao có sẵn, miễn phí, đồng thời IFrame API vẫn cung cấp đầy đủ sự kiện cần thiết để em xây dựng luồng "xem hết video → tự động chuyển quiz" mà không cần nút "Đã xem xong" thủ công.

3.7 Thanh toán trực tuyến: VNPay

Để đáp ứng yêu cầu hỗ trợ khóa học có phí (Chương 2), em tích hợp cổng thanh toán VNPay [11] – cổng thanh toán nội địa phổ biến tại Việt Nam, sử dụng môi trường sandbox cho mục đích phát triển và kiểm thử. Quy trình thanh toán em thiết kế gồm hai bước: (i) backend tạo URL thanh toán có chữ ký HMAC-SHA512 dựa trên thông tin giao dịch và mã bí mật (*hash secret*); (ii) sau khi người dùng thanh toán, VNPay gọi lại endpoint `/api/v1/payments/vnpay-return`, backend xác thực lại chữ ký trước khi ghi nhận giao dịch thành công và đăng ký khóa học cho người dùng.

Lựa chọn thay thế là các cổng thanh toán quốc tế như Stripe hoặc PayPal. Tuy nhiên các cổng này yêu cầu tài khoản doanh nghiệp đã xác minh và chủ yếu hỗ trợ thẻ quốc tế, không phù hợp với đối tượng người dùng Việt Nam mà hệ thống hướng tới. Em chọn VNPay sandbox vì cho phép thử nghiệm đầy đủ luồng thanh toán mà không cần tài khoản doanh nghiệp thật, phù hợp với phạm vi đồ án tốt nghiệp.

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

Chương này trình bày toàn bộ quá trình em phân tích thiết kế, triển khai và đánh giá hệ thống NihongoFlow, dựa trên các yêu cầu đã xác định ở Chương 2 và các công nghệ đã lựa chọn ở Chương 3.

4.1 Thiết kế kiến trúc

4.1.1 Lựa chọn kiến trúc phần mềm

Em thiết kế hệ thống theo kiến trúc ba lớp kết hợp với mô hình phân tách frontend-backend qua RESTful API. Phía backend, em tổ chức ba lớp như sau:

- **Lớp Controller** (gói `controller`): tiếp nhận request HTTP, thực hiện validate đầu vào (`@Valid`) và ánh xạ sang DTO; không chứa logic nghiệp vụ.
- **Lớp Service** (gói `service`): chứa toàn bộ logic nghiệp vụ – tính điểm quiz, kiểm tra điều kiện đăng ký khóa học, cập nhật streak, xác thực chữ ký VNPay, v.v.
- **Lớp Repository** (gói `repository`): các interface kế thừa `JpaRepository`, chỉ chứa định nghĩa truy vấn (kể cả truy vấn tùy biến bằng `@Query`), không chứa logic.

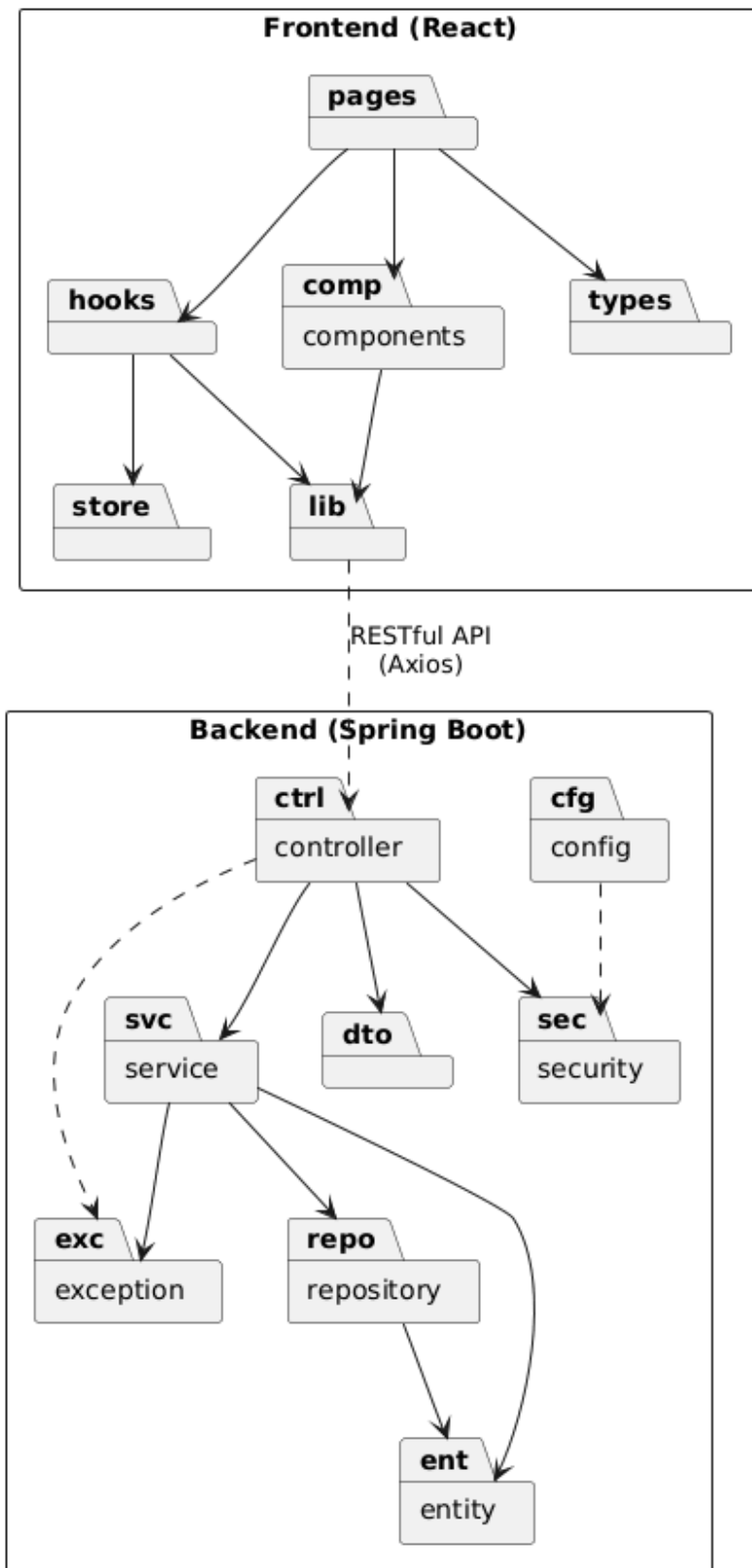
Bên cạnh ba lớp chính, em bổ sung các gói hỗ trợ: `entity` (các thực thể JPA ánh xạ tới bảng CSDL), `dto` (đối tượng truyền dữ liệu cho request/response), `security` (bộ lọc JWT, cấu hình `UserDetailsService`), `config` (cấu hình Spring, CORS, VNPay), và `exception` (xử lý lỗi tập trung qua `@RestControllerAdvice` và `ApiException`).

P phía frontend, em tổ chức kiến trúc tương ứng theo các tầng: `Page` (hiển thị giao diện, điều hướng), `Hook` (gói `hooks` – toàn bộ `useQuery/useMutation` bằng TanStack Query), `Lib` (gói `lib` – hàm thuần, hằng số, instance Axios), và `Component` (gói `components` – thành phần giao diện tái sử dụng). Em quy định `Page` chỉ được phép gọi dữ liệu thông qua hook, không gọi trực tiếp Axios; hook không chứa state giao diện (như trạng thái mở/đóng modal).

So với mô hình MVC truyền thống, kiến trúc trên có thể coi lớp Controller + Service như phần "Controller" mở rộng, lớp Entity/Repository như "Model", và toàn bộ ứng dụng React như "View" – giao tiếp với "Model/Controller" hoàn toàn qua API JSON thay vì render phía server.

4.1.2 Thiết kế tổng quan

Hình 4.1 mô tả biểu đồ gói của toàn bộ hệ thống, bao gồm các gói backend (controller, service, repository, entity, dto, security, config, exception) và các gói frontend (pages, hooks, lib, components, store, types). Em thiết kế quan hệ phụ thuộc tuân theo nguyên tắc một chiều: controller phụ thuộc service, service phụ thuộc repository và entity; không có chiều phụ thuộc ngược lại. Tương tự, pages phụ thuộc hooks và components, hooks phụ thuộc lib và store, không có gói tầng dưới phụ thuộc ngược lên tầng trên.



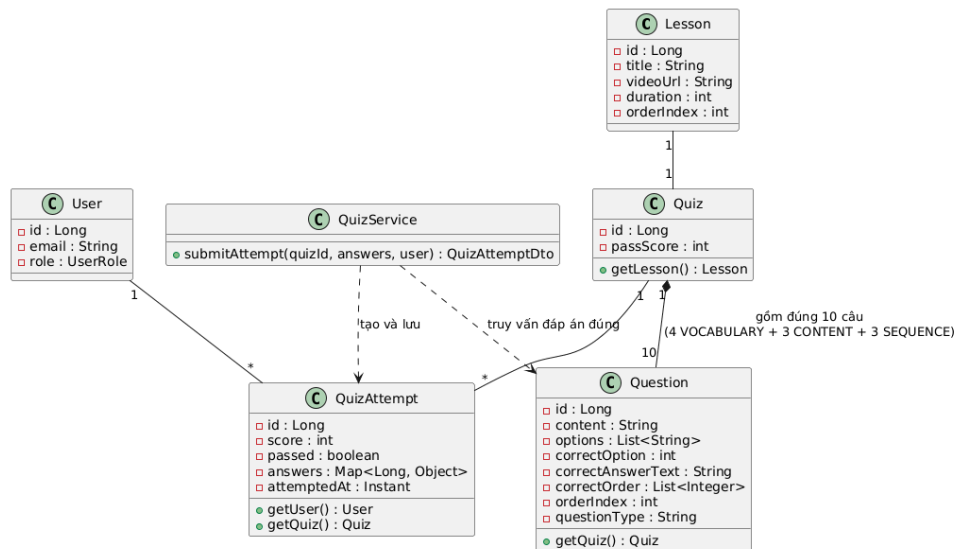
Hình 4.1: Biểu đồ gói tổng quan hệ thống NihongoFlow

Hai chiều giao tiếp chính giữa frontend và backend là: (i) các request CRUD/ng nghiệp vụ qua `lib/axios.ts` (instance Axios duy nhất, có interceptor tự động đính kèm access token và làm mới token khi hết hạn); (ii) request callback từ VNPay tới

4.1.3 Thiết kế chi tiết gói

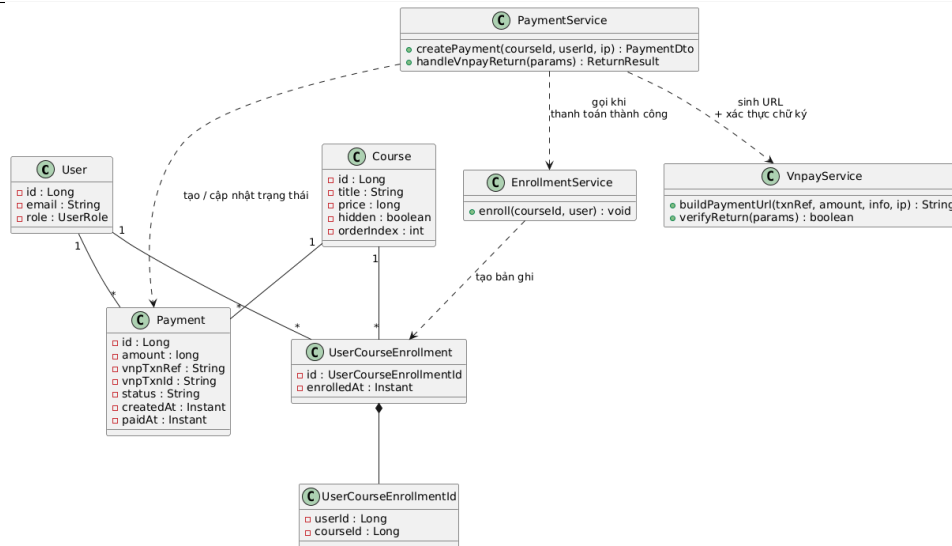
Hai nhóm package em đánh giá có quan hệ phức tạp nhất trong hệ thống là nhóm Quiz và nhóm Enrollment, được em trình bày chi tiết dưới đây.

Nhóm Quiz. Hình 4.2 mô tả các lớp Quiz, Question, QuizAttempt và quan hệ giữa chúng. Một Lesson có quan hệ 1–1 với Quiz; một Quiz kết hợp (composition) với nhiều Question (đúng 10 câu, chia theo questionType); mỗi QuizAttempt liên kết tới một Quiz và một User, lưu kết quả dưới dạng cột JSON answers. QuizService phụ thuộc (dependency) vào cả ba repository tương ứng để thực hiện việc chấm điểm.



Hình 4.2: Biểu đồ lớp nhóm Quiz

Nhóm Enrollment. Hình 4.3 mô tả quan hệ giữa User, Course và UserCourseEnrollment – thực thể trung gian biểu diễn quan hệ nhiều–nhiều giữa người dùng và khóa học, có khóa chính phức hợp (userId, courseId). Payment liên kết với User và Course, được PaymentService sử dụng để tạo và xác thực giao dịch VNPay; khi giao dịch thành công, PaymentService gọi EnrollmentService (kết hợp/association) để tạo bản ghi UserCourseEnrollment.



Hình 4.3: Biểu đồ lớp nhóm Enrollment và Payment

4.2 Thiết kế chi tiết

4.2.1 Thiết kế giao diện

Em thiết kế giao diện hệ thống hướng tới máy tính để bàn, độ phân giải tham chiếu 1920×1080 , với bố cục nội dung giới hạn chiều rộng tối đa 1120px, căn giữa. Em lựa chọn phong cách thiết kế tối giản, nền tối, lấy cảm hứng từ Resend.com, với bộ màu chuẩn hóa qua biến CSS:

- **Nền:** `-bg` (#0a0a0a), `-s1` (#111111) cho thẻ/khối nội dung, `-s2` (#161616) cho khối lồng bên trong
- **Viền:** `-b1` (#262626), `-b2` (#333333)
- **Chữ:** `-t1` (#ededed) chữ chính, `-t2` (#a1a1a1) chữ phụ, `-t3` (#555555) chữ mờ
- **Nhấn:** `-acc` (#e94560) – duy nhất một màu nhấn cho toàn hệ thống, dùng cho nút hành động chính, thanh tiến độ và trạng thái được chọn

Phông chữ chính em chọn là Inter cho văn bản tiếng Việt/tiếng Anh và Noto Sans JP cho văn bản tiếng Nhật (cỡ chữ tối thiểu 14px, độ đậm tối thiểu 500 để đảm bảo hiển thị rõ nét). Khoảng cách giữa các phần tử tuân theo lưới 8px. Các thành phần dùng chung (nút, thẻ, huy hiệu, ô nhập liệu) em xây dựng từ shadcn/ui và tùy biến lại theo bộ token trên thông qua `globals.css`, đảm bảo không có giá trị màu hay khoảng cách "hard-code" rải rác trong mã nguồn.

Một số màn hình tiêu biểu minh họa các nguyên tắc trên gồm trang chủ (Hình 4.7), trang khóa học, trang xem video kết hợp quiz, và khu vực quản trị, được em trình bày chi tiết tại Mục 4.3.3.

4.2.2 Thiết kế lớp

Phần này em trình bày chi tiết ba lớp chủ đạo của hệ thống: `Question`, `QuizAttempt`, và `UserCourseEnrollment`.

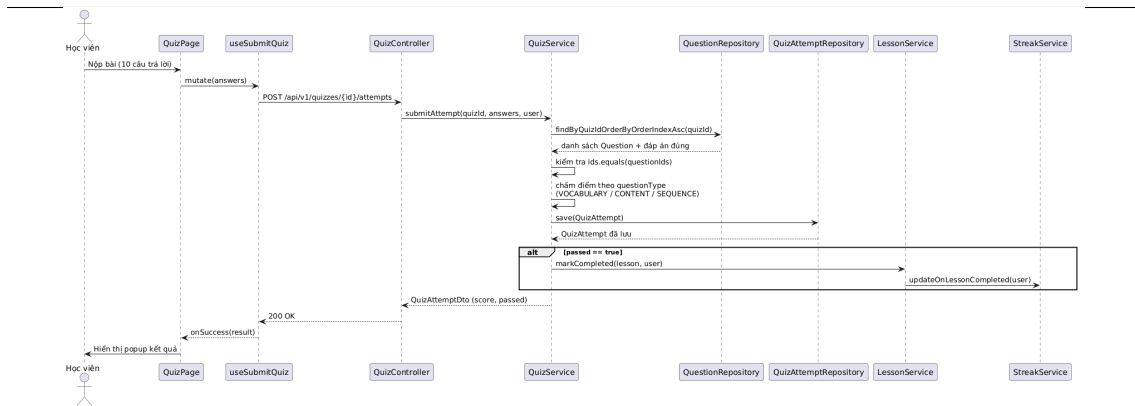
Lớp `Question` đại diện cho một câu hỏi trong quiz, với các thuộc tính: `id` (Long), `quiz` (quan hệ ManyToOne tới Quiz), `content` (String – nội dung câu hỏi), `options` (kiểu JSON, mảng 4 chuỗi đáp án), `correctOption` (int – chỉ số đáp án đúng từ 0–3, chỉ áp dụng cho loại CONTENT), `correctAnswerText` (String – đáp án văn bản đúng, chỉ áp dụng cho loại VOCABULARY), `correctOrder` (kiểu JSON – thứ tự đúng, chỉ áp dụng cho loại SEQUENCE), `orderIndex` (int – thứ tự câu hỏi từ 1–10), và `questionType` (enum: VOCABULARY, CONTENT, SEQUENCE). Thiết kế tổng quát hóa qua `questionType` cho phép em thêm dạng câu hỏi mới trong tương lai mà không cần thay đổi schema – nội dung này em phân tích sâu hơn ở Mục 5.1.

Lớp `QuizAttempt` lưu một lượt làm bài, gồm: `id`, `user` (ManyToOne tới User), `quiz` (ManyToOne tới Quiz), `score` (int), `passed` (boolean), `answers` (kiểu JSON – ánh xạ `questionId` sang đáp án đã chọn, kiểu dữ liệu của đáp án phụ thuộc `questionType` của từng câu), và `attemptedAt` (thời điểm nộp bài). Phương thức nghiệp vụ chính `QuizService.submitAttempt(...)` nhận vào `quizId`, `answers` và người dùng hiện tại, thực hiện: (i) kiểm tra tập `questionId` gửi lên khớp chính xác (đối xứng hai chiều) với tập câu hỏi của quiz; (ii) so sánh từng đáp án với đáp án đúng tương ứng theo `questionType` để tính số câu đúng; (iii) so sánh với `Quiz.passScore` để xác định `passed`; (iv) lưu `QuizAttempt` và gọi `LessonService.markCompleted(...)` nếu `passed = true`, qua đó cập nhật streak nếu đây là lượt hoàn thành bài học đầu tiên trong ngày.

Lớp `UserCourseEnrollment` là thực thể có khóa chính phức hợp (`userId`, `courseId`), gồm thêm thuộc tính `enrolledAt`. Phương thức `PaymentService.createPayment(...)` kiểm tra điều kiện: nếu `Course.price == 0`, gọi trực tiếp `EnrollmentService.enroll(...)` để tạo bản ghi enrollment; nếu `price > 0`, việc enroll chỉ được thực hiện sau khi Payment được xác nhận thành công qua luồng VNPay.

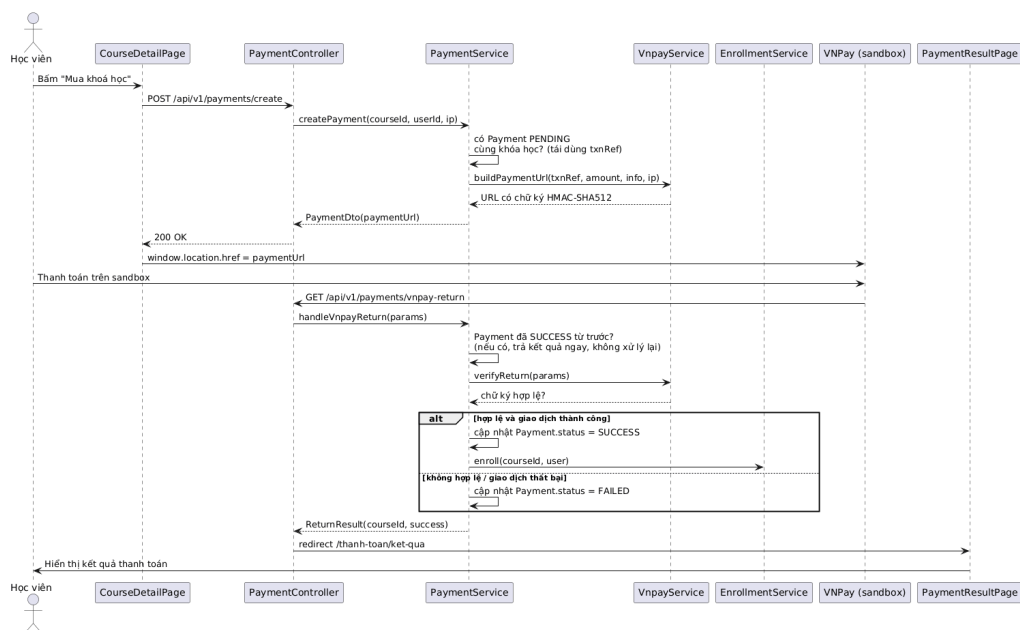
Hai luồng tương tác em đánh giá là quan trọng nhất được minh họa bằng biểu đồ trình tự:

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG



Hình 4.4: Biểu đồ trình tự: Nộp bài quiz

Hình 4.4 mô tả luồng nộp bài quiz: QuizPage (frontend) gọi hook useSubmitQuiz, gửi `POST /api/v1/quizzes/:id/attempts` tới QuizController; QuizController gọi `QuizService.submitAttempt`, dịch vụ này truy vấn `QuestionRepository` để lấy đáp án đúng, tính điểm, lưu `QuizAttempt` qua `QuizAttemptRepository`, đồng thời gọi `LessonService/StreakService` để cập nhật trạng thái hoàn thành bài học và chuỗi ngày học; kết quả (điểm số, đạt/không đạt) được trả về và hiển thị qua popup kết quả.



Hình 4.5: Biểu đồ trình tự: Đăng ký khóa học có phí qua VNPAY

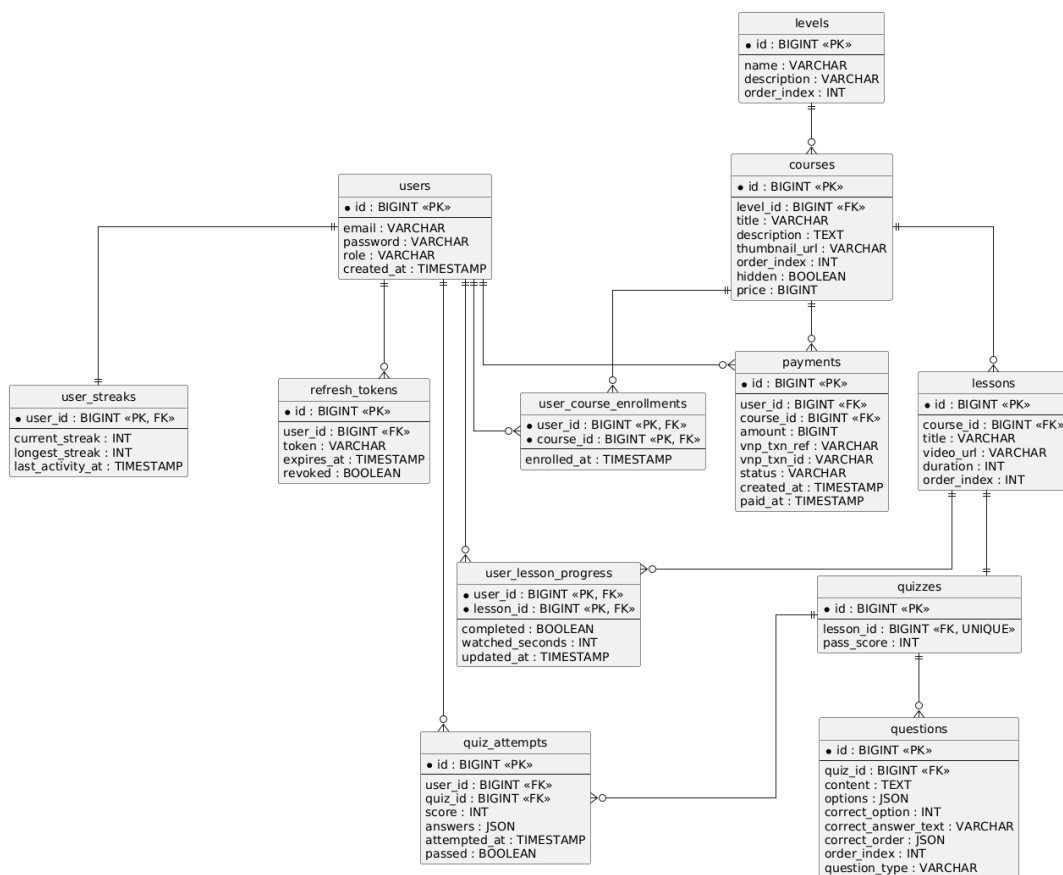
Hình 4.5 mô tả luồng đăng ký khóa học có phí: CourseDetailPage gọi `useCreatePaymentMutation`, gửi `POST /api/v1/payments/create` tới PaymentController; PaymentService tạo bản ghi `Payment` ở trạng thái chờ, gọi `VnpayService` để sinh URL thanh toán kèm chữ ký HMAC-SHA512, và trả URL về cho frontend để chuyển hướng (`window.location.href`). Sau

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

khi người dùng thanh toán trên cổng VNPay, VNPay gọi lại GET/api/v1/payments/vnpay-return; VnpayService xác thực lại chữ ký, PaymentService cập nhật trạng thái Payment và gọi EnrollmentService.enroll(...) nếu thành công, sau đó PaymentController chuyển hướng (redirect) trình duyệt về trang /thanh-toan/ket-qua của frontend.

4.2.3 Thiết kế cơ sở dữ liệu

Hình 4.6 trình bày biểu đồ thực thể liên kết (ERD) của toàn bộ hệ thống, gồm 12 bảng chính: users, levels, courses, lessons, quizzes, questions, quiz_attempts, user_lesson_progress, user_course_enrollments, user_course_enrollments, user_streaks, payments, và bảng lưu refresh token.



Hình 4.6: Biểu đồ thực thể liên kết (ERD) hệ thống NihongoFlow

Một số điểm thiết kế em lưu ý đặc biệt khi xây dựng schema:

- **Khóa chính phức hợp:** user_lesson_progress dùng khóa chính phức hợp (user_id, lesson_id); user_course_enrollments dùng khóa chính phức hợp (user_id, course_id). Thiết kế này đảm bảo mỗi người dùng chỉ có đúng một bản ghi tiến độ cho mỗi bài học và đúng một bản ghi đăng ký cho mỗi khóa học.

- **Cột kiểu JSON:** `questions.options` (mảng 4 chuỗi đáp án), `questions.correct_order` (thứ tự đúng cho câu trình tự) và `quiz_attempts.answers` (ánh xạ `questionId` sang đáp án đã chọn) được em lưu dưới dạng JSON, cho phép linh hoạt số lượng và cấu trúc mà không cần bảng phụ.
- **Phân cấp nội dung:** quan hệ một–nhiều theo chuỗi `levels → courses → lessons → quizzes → questions`, mỗi bảng con có khóa ngoại trỏ tới bảng cha và cột `order/order_index` để xác định thứ tự hiển thị.
- **Ràng buộc xóa lan truyền (ON DELETE CASCADE):** khi một `lesson` hoặc `course` bị xóa, các bản ghi liên quan trong các bảng `quizzes`, `questions`, `quiz_attempts`, `user_lesson_progress` và `user_course_enrollments` được tự động xóa theo, tránh dữ liệu mồ côi.
- **Quản lý phiên bản schema:** toàn bộ cấu trúc CSDL được em quản lý qua Flyway, cho phép áp dụng schema một cách có kiểm soát và tái lập trên nhiều môi trường. So với một schema quan hệ tối thiểu, lược đồ của hệ thống được em mở rộng thêm: bảng `user_course_enrollments` (đăng ký khóa học), cờ `hidden` và cột `price` trên `courses` (ẩn/hiện và định giá khóa học), bảng `payments` (lịch sử thanh toán), cột `question_type` cùng hai cột đáp án riêng `correct_answer_text/correct_order` trên `questions` (hỗ trợ ba dạng câu hỏi VOCABULARY/CONTENT/SEQUENCE với cách chấm điểm khác nhau – xem Mục 5.1), và ràng buộc ON DELETE CASCADE đã nêu ở trên.

4.3 Xây dựng ứng dụng

4.3.1 Thư viện và công cụ sử dụng

Bảng 4.1 liệt kê các công cụ và thư viện chính em sử dụng để xây dựng hệ thống.

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

Thành phần	Công cụ / Thư viện	Phiên bản
Ngôn ngữ backend	Java	21
Framework backend	Spring Boot	3.x
Bảo mật	Spring Security + JWT	3.x / 0.12.x
ORM	Spring Data JPA (Hibernate)	6.x
CSDL	MySQL	8.0
Quản lý migration	Flyway	10.x
Build tool backend	Maven	3.9.x
Ngôn ngữ frontend	TypeScript (strict mode)	5.x
Framework frontend	React + Vite	React 18 / Vite 5
Quản lý server state	TanStack Query	5.x
Quản lý client state	Zustand	4.x
Định tuyến	React Router	6.x
CSS Framework	Tailwind CSS + shadcn/ui	4.x
HTTP client	Axios	1.x
Phát video	YouTube IFrame Player API	–
Thanh toán	VNPay Sandbox	–
Kiểm thử frontend	Vitest + Testing Library	4.x
Kiểm thử backend	JUnit 5 + Mockito (qua spring-boot-starter-test)	5.x
Container hóa môi trường dev	Docker Compose (MySQL)	–

Bảng 4.1: Danh sách công cụ và thư viện sử dụng

4.3.2 Kết quả đạt được

Em đã xây dựng và chạy được đầy đủ hai luồng người dùng (học viên và quản trị viên) trên môi trường phát triển cục bộ. Bảng 4.2 thống kê quy mô mã nguồn theo từng tầng tại thời điểm hoàn thiện đồ án.

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

Phía	Thành phần	Số lượng
Backend	Controller	10
	Service	14
	Repository	12
	Entity	15
	DTO	27
	Lớp Security	6
	Lớp Config	5
	Lớp Exception	3
Frontend	Page	13
	Custom hook	11
	Lib util	5
	Component dùng chung	11
	Component theo domain (quiz, dashboard)	9

Bảng 4.2: Thống kê quy mô mã nguồn theo tầng

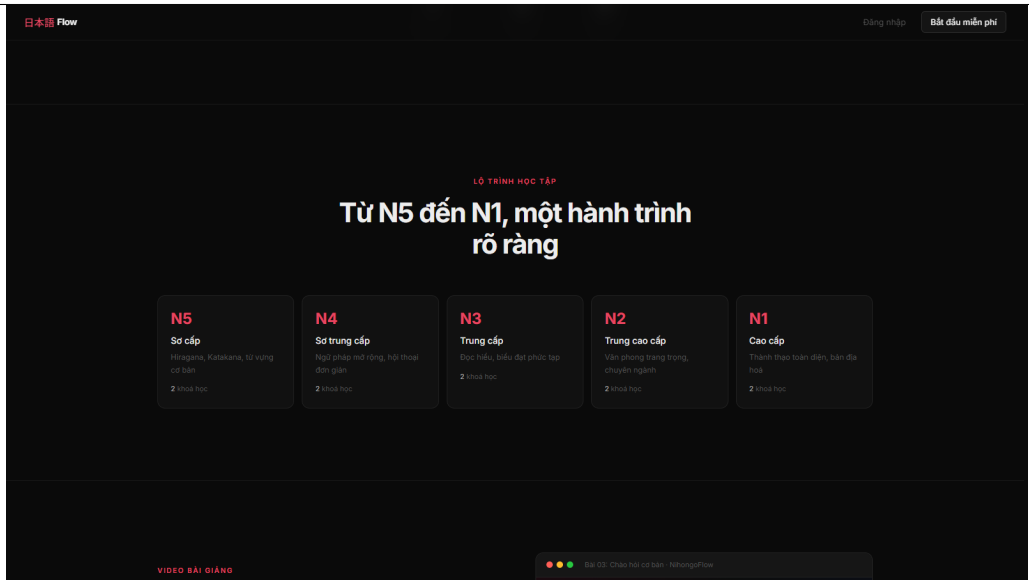
Dữ liệu mẫu cho môi trường phát triển bao gồm 5 cấp độ (N5–N1), 10 khóa học (2 khóa/cấp độ), 24 bài học, 24 quiz và 240 câu hỏi (mỗi quiz đúng 10 câu, chia 4 câu VOCABULARY, 3 câu CONTENT, 3 câu SEQUENCE).

Về mặt chức năng, em đã hoàn thiện: xác thực JWT (đăng ký, đăng nhập, làm mới token, đăng xuất); quản lý nội dung theo cấu trúc Cấp độ – Khóa học – Bài học – Quiz; đăng ký khóa học (miễn phí và có phí qua VNPay); phát video YouTube với theo dõi tiến độ và tự động chuyển sang quiz khi xem hết video lần đầu; chấm điểm và lưu lịch sử làm quiz kèm chức năng xem lại đáp án theo từng loại câu hỏi; theo dõi streak học tập theo ngày; và toàn bộ chức năng quản trị (CRUD khóa học/bài học, quản lý câu hỏi quiz).

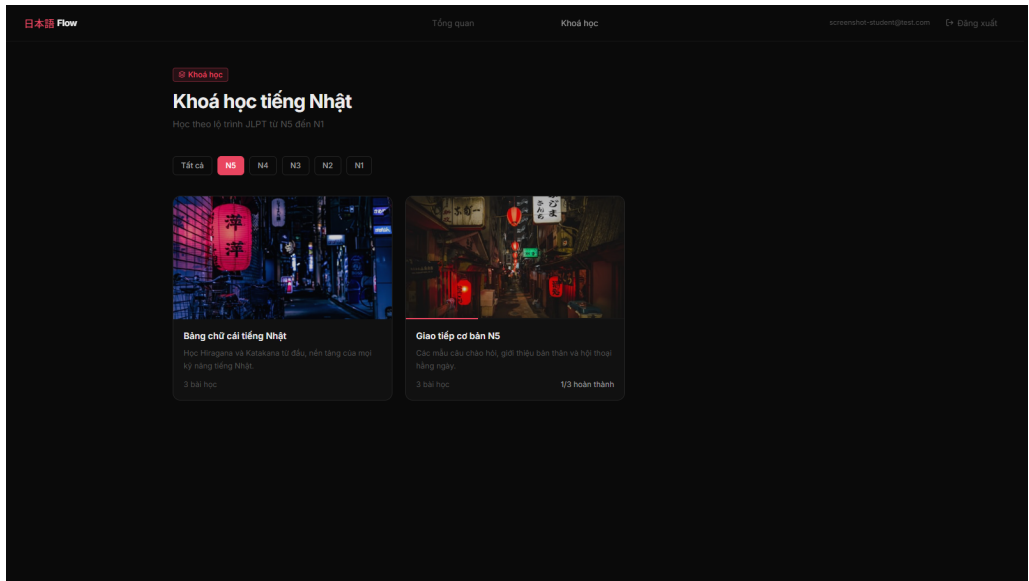
4.3.3 Minh họa các chức năng chính

Phần này trình bày ảnh chụp màn hình của các chức năng chính trong hệ thống.

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

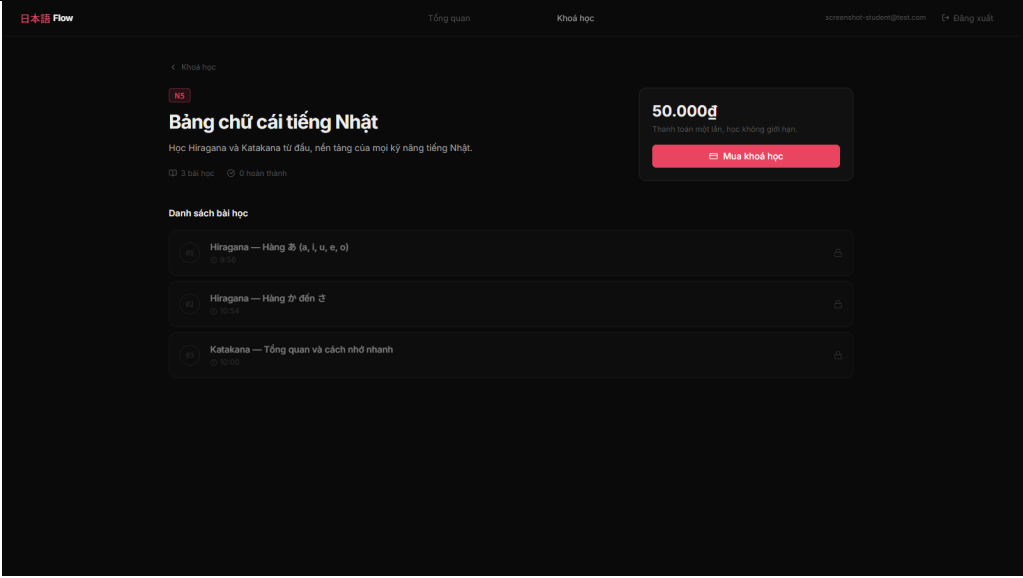


Hình 4.7: Trang chủ – giới thiệu các cấp độ N5–N1

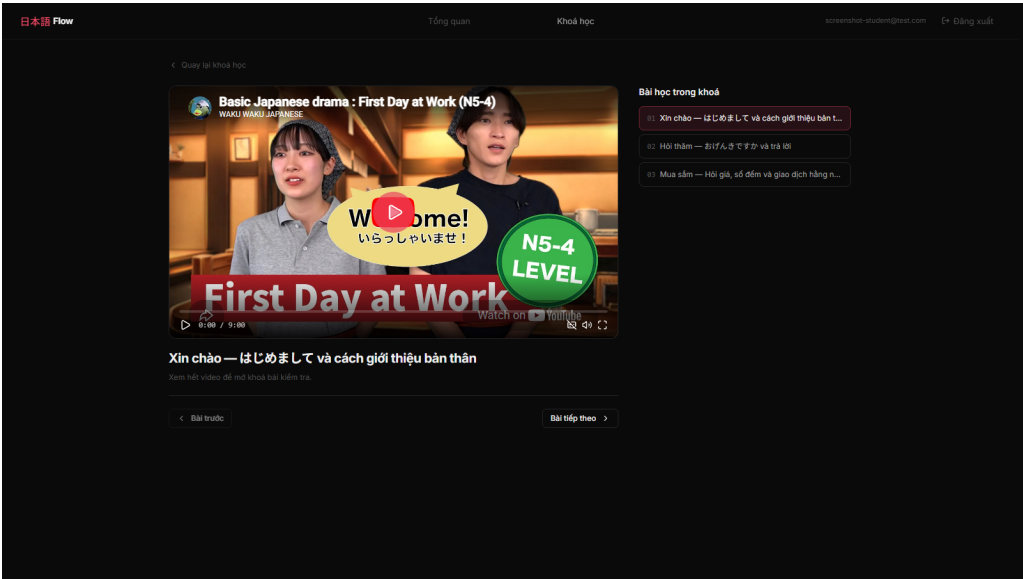


Hình 4.8: Trang danh sách khóa học, lọc theo cấp độ

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG



Hình 4.9: Trang chi tiết khóa học – danh sách bài học và nút đăng ký/mua khóa học



Hình 4.10: Trang xem video bài học với YouTube custom player

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

日本語 Flow

Tổng quan Khoá học screenshot-student@test.com Đăng xuất

← Quay lại khoá học

Kiểm tra bài học

Xin chào — はじめまして và cách giới thiệu bản thân

Từ vựng 4 câu hỏi

❓ VÍ DỤ MINH HỌA

Ví dụ: Chữ 「文」 đọc là gì?

ひ

1. "はじめまして" nghĩa là gì?

Nhập câu trả lời...

2. "わたし" nghĩa là gì?

Nhập câu trả lời...

3. "です" có vai trò gì trong câu?

Nhập câu trả lời...

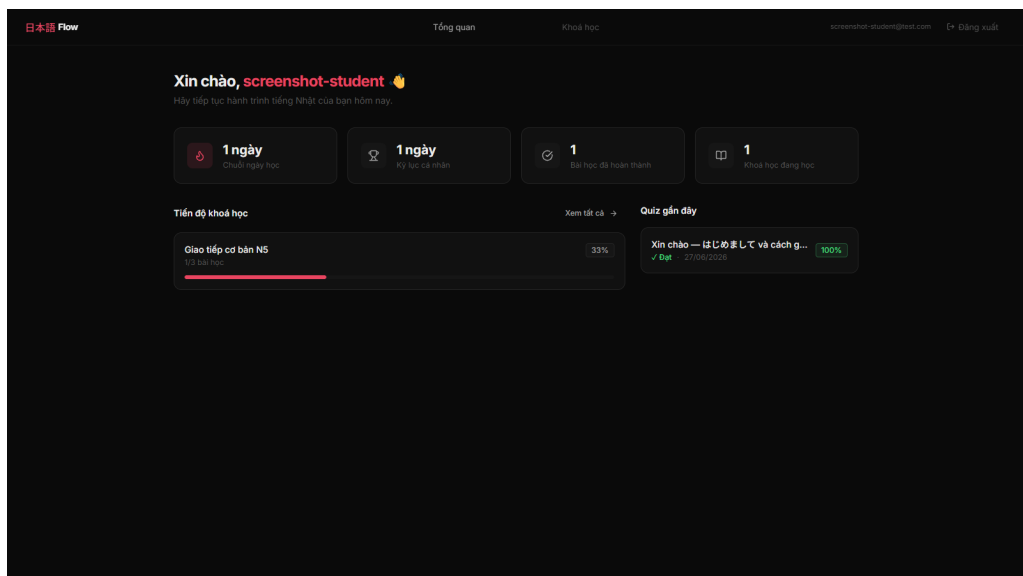
4. "どうぞよろしく" nghĩa là gì?

Nhập câu trả lời...

Tiếp theo →

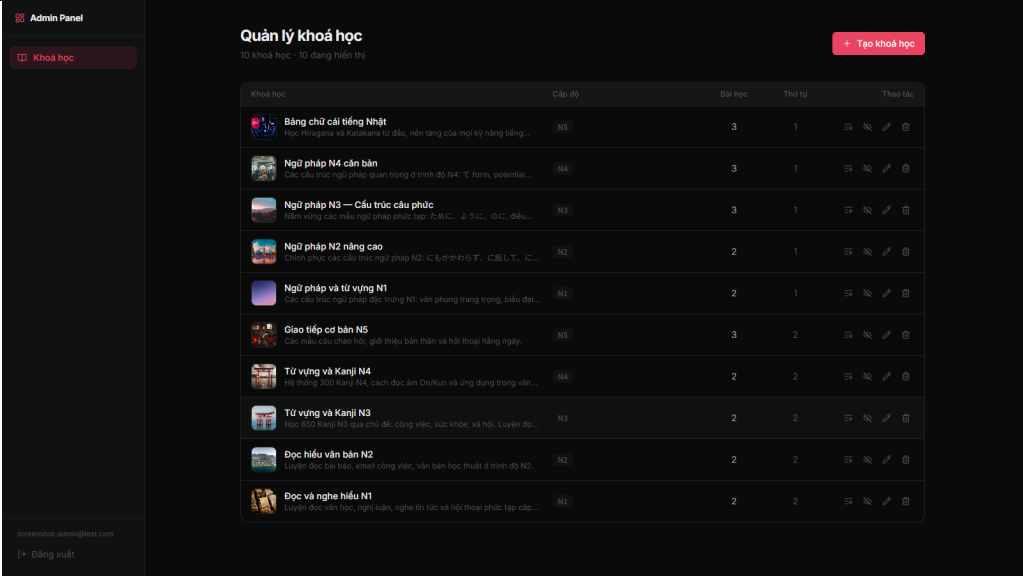
Vui lòng trả lời tất cả 4 câu hỏi trước khi tiếp tục.

Hình 4.11: Trang làm bài quiz theo wizard 3 bước, chia theo 3 nhóm câu hỏi

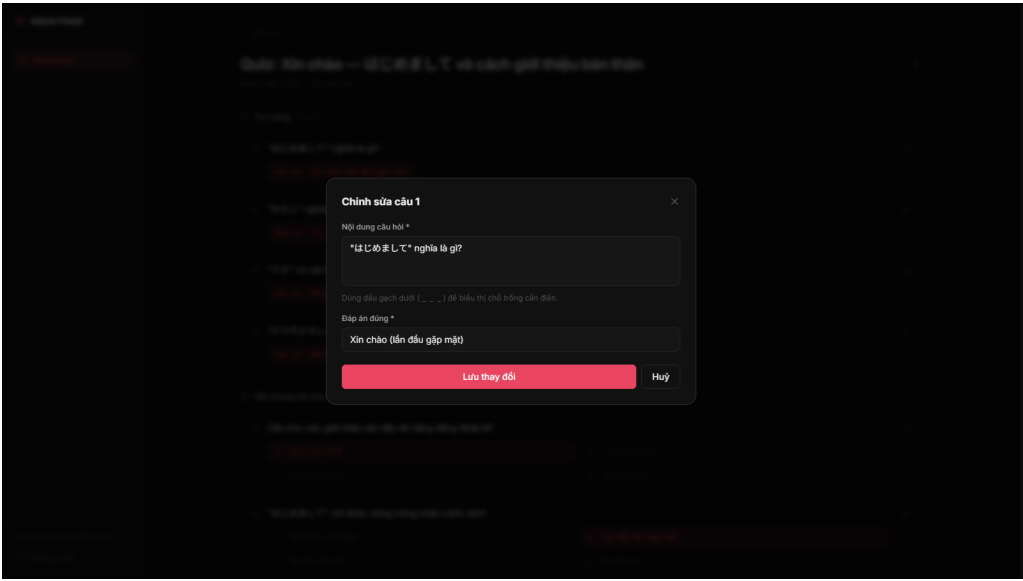


Hình 4.12: Trang Dashboard – tiến độ học tập, streak và lịch sử làm quiz

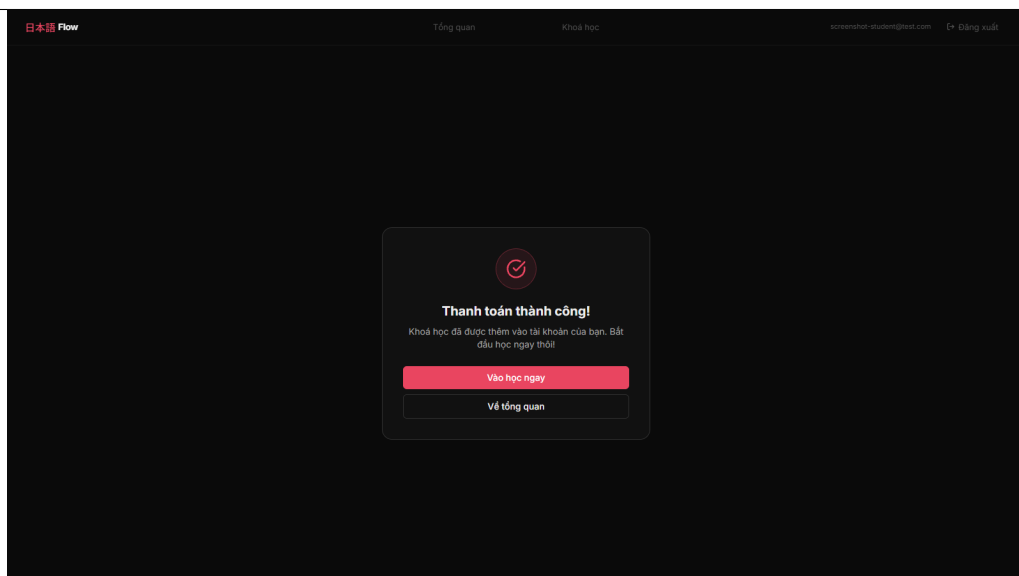
CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG



Hình 4.13: Trang quản trị – danh sách và quản lý khóa học



Hình 4.14: Trang quản trị – chỉnh sửa câu hỏi quiz



Hình 4.15: Trang kết quả thanh toán VNPay

4.4 Kiểm thử

Em kiểm thử hệ thống theo hai hình thức bổ trợ nhau: kiểm thử tự động (Vitest cho các logic và luồng quan trọng phía frontend, JUnit 5 + Mockito cho logic chấm điểm quiz phía backend), và kiểm thử thủ công theo kịch bản cho các luồng nghiệp vụ cần dữ liệu thực và môi trường tích hợp đầy đủ (cơ sở dữ liệu, cổng thanh toán sandbox).

Bộ kiểm thử tự động phía frontend (77 trường hợp) gồm các trường hợp kiểm tra logic phát video (debounce lưu tiến độ, resume từ vị trí đã xem, không điều hướng lại khi xem lại bài đã hoàn thành), logic của quiz wizard (sắp xếp và nhóm câu hỏi theo loại, điều kiện đã trả lời đủ câu mới cho phép sang trang tiếp theo, tính điểm và xác định đạt/không đạt, đúng kiểu dữ liệu đáp án theo từng loại câu hỏi khi render trực tiếp các component `VocabularyStep/ContentStep`), và các hàm tiện ích dùng chung (định dạng ngày giờ, định dạng thời lượng video). Phía backend, em bổ sung 12 trường hợp kiểm thử tự động cho `QuizService`, tập trung vào logic chấm điểm riêng theo từng loại câu hỏi (so khớp văn bản cho `VOCABULARY`, so khớp chỉ số đáp án cho `CONTENT`, so khớp thứ tự cho `SEQUENCE`) và các trường hợp biên khi tính `passScore` (ví dụ: đúng 7/10 câu với ngưỡng 70% phải đạt, đúng 6/10 câu phải không đạt). Toàn bộ các trường hợp này đều cho kết quả đạt khi em chạy lại bộ kiểm thử lần cuối trước khi hoàn thiện báo cáo.

Đối với các luồng nghiệp vụ cần môi trường tích hợp đầy đủ, em kiểm thử thủ công trên hai chức năng quan trọng nhất: nộp bài quiz và đăng ký khóa học có phí qua VNPay. Bảng 4.3 và Bảng 4.4 trình bày các trường hợp kiểm thử tiêu biểu.

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

Mã TC	Đầu vào	Kết quả mong đợi	Kết quả
TC-Quiz-01	Nộp đủ 10 câu, tất cả đáp án đúng	score = 100, passed = true	Đạt
TC-Quiz-02	Nộp đủ 10 câu, số câu đúng dưới ngưỡng passScore	passed = false, hiển thị nút "Làm lại"	Đạt
TC-Quiz-03	Nộp thiếu một questionId so với quiz	API trả lỗi VALIDATION_ERROR	Đạt
TC-Quiz-04	Nộp dư một questionId không thuộc quiz	API trả lỗi VALIDATION_ERROR	Đạt
TC-Quiz-05	Lượt làm đầu tiên trong ngày, kết quả đạt	UserStreak.currentStreak tăng thêm 1	Đạt

Bảng 4.3: Trường hợp kiểm thử chức năng nộp bài quiz

Mã TC	Đầu vào	Kết quả mong đợi	Kết quả
TC-Pay-01	Đăng ký khóa học price = 0	Tạo bản ghi UserCourse Enrollment ngay, không qua VNPay	Đạt
TC-Pay-02	Đăng ký khóa học price > 0, thanh toán thành công trên sandbox	Callback xác thực HMAC-SHA512 hợp lệ, tạo Payment thành công và UserCourseEnrollment	Đạt
TC-Pay-03	Hủy giao dịch trên trang VNPay	Payment ở trạng thái thất bại, không tạo enrollment, frontend hiển thị thông báo thất bại	Đạt
TC-Pay-04	Callback với chữ ký không hợp lệ (giả mạo)	API từ chối, không cập nhật trạng thái Payment	Đạt
TC-Pay-05	VNPay gọi lại callback lần thứ hai cho giao dịch đã SUCCESS	Backend trả kết quả thành công ngay, không enroll lại, không ghi đè dữ liệu giao dịch gốc	Đạt

Bảng 4.4: Trường hợp kiểm thử chức năng đăng ký khóa học và thanh toán VNPay

Tổng cộng 10 trường hợp kiểm thử thử công nêu trên đều cho kết quả đạt. Các trường hợp kiểm thử bổ sung cho những chức năng khác (xác thực JWT, enrollment khóa học miễn phí, CRUD nội dung phía quản trị) được em thực hiện theo phương pháp tương tự trong quá trình phát triển, không trình bày chi tiết trong báo cáo.

4.5 Triển khai

Em triển khai và kiểm thử hệ thống trên môi trường phát triển cục bộ (localhost), gồm ba thành phần chạy song song:

- **Cơ sở dữ liệu MySQL:** chạy qua Docker Compose (`docker compose up -d mysql`), cổng mặc định 3306, dữ liệu được khởi tạo qua migration Flyway và nạp dữ liệu mẫu bằng `mysql.exe`.
- **Backend Spring Boot:** chạy bằng `mvn spring-boot:run`, lắng nghe ở cổng 8080, cung cấp toàn bộ API dưới tiền tố `/api/v1`.
- **Frontend React (Vite):** chạy bằng `npm run dev`, lắng nghe ở cổng 5173, gọi API backend qua biến môi trường `VITE_API_BASE_URL`.

Cấu hình nhảy cảm (chuỗi kết nối CSDL, khóa bí mật JWT, thông tin sandbox VNPay) được em quản lý qua biến môi trường, không hard-code trong mã nguồn.

CHƯƠNG 5. CÁC GIẢI PHÁP VÀ ĐÓNG GÓP NỔI BẬT

Chương này trình bày sáu giải pháp và đóng góp em đầu tư nhiều công sức nhất trong quá trình thực hiện đồ án. Mỗi mục được em trình bày theo cấu trúc: dẫn dắt vấn đề (gắn với yêu cầu đã nêu ở Chương 2), giải pháp em đã thiết kế và triển khai, và kết quả đạt được.

5.1 Thiết kế hệ thống quiz mở rộng theo kiểu câu hỏi

Vấn đề. Mục 2.4 (yêu cầu phi chức năng) chỉ ra rằng hệ thống quiz cần hỗ trợ nhiều dạng câu hỏi khác nhau phù hợp với đặc thù học tiếng Nhật (từ vựng, nội dung video, trình tự sự kiện), và phải có khả năng mở rộng thêm dạng câu hỏi mới (ví dụ nghe-chọn đáp án, nối cặp) trong tương lai mà không phá vỡ hệ thống hiện có. Một thiết kế CSDL "cứng" – ví dụ tạo riêng một bảng cho mỗi dạng câu hỏi – sẽ khiến việc thêm dạng câu hỏi mới đòi hỏi thay đổi schema, viết lại truy vấn, và sửa đổi cả hai phía backend lẫn frontend.

Giải pháp. Như em đã giới thiệu ở Mục 4.2.2, em thiết kế bảng `questions` tổng quát với bốn thành phần cốt lõi:

- Cột `question_type` (enum: `VOCABULARY`, `CONTENT`, `SEQUENCE`) phân loại câu hỏi theo mục đích sử dụng, được em thêm vào schema mà không ảnh hưởng tới dữ liệu hiện có.
- Cột `options` kiểu JSON, lưu mảng đáp án dưới dạng chuỗi – cho phép số lượng và nội dung đáp án linh hoạt mà không cần thêm cột.
- Ba cột đáp án đúng riêng theo loại câu hỏi: `correct_option` (chỉ số đáp án đúng, dùng cho `CONTENT`), `correct_answer_text` (đáp án văn bản, dùng cho `VOCABULARY`), và `correct_order` (thứ tự đúng dạng JSON, dùng cho `SEQUENCE`) – em bổ sung hai cột sau vào schema khi cần hỗ trợ chấm điểm cho hai loại câu hỏi mới.
- Cột `order_index` xác định vị trí câu hỏi trong quiz (1–10), được em dùng để nhóm và hiển thị câu hỏi theo từng trang trong wizard làm quiz ở frontend.

Phía backend, em thiết kế `QuizService.submitAttempt(...)` chấm điểm bằng cách rẽ nhánh theo `questionType` của từng câu: so khớp văn bản (đã chuẩn hóa khoảng trắng và chữ hoa/thường) với `VOCABULARY`, so khớp chỉ số đáp án với `CONTENT`, và so khớp toàn bộ thứ tự với `SEQUENCE`. Việc kiểm tra tính hợp lệ của bộ câu trả lời em sử dụng so sánh đối xứng hai chiều `ids.equals(questionIds)` – đảm bảo tập câu hỏi gửi lên không thiếu và không thừa so với quiz, thay vì chỉ kiểm tra một chiều bằng `containsAll` (vốn

cho phép bỏ sót câu hỏi mà vẫn được coi là hợp lệ). Phía frontend, em định nghĩa `QuestionType` là một union type TypeScript, và bảng tra cứu `QUESTION_SECTION_META` (đặt tại `src/lib/quiz-constants.ts`) ánh xạ mỗi `questionType` sang nhãn hiển thị và icon tương ứng – khi cần thêm một dạng câu hỏi mới, em chỉ cần bổ sung một entry vào bảng tra cứu này, thêm giá trị enum tương ứng ở backend, và viết thêm một component hiển thị riêng cho dạng câu hỏi đó.

Kết quả. Cấu trúc 10 câu hỏi/quiz (4 VOCABULARY + 3 CONTENT + 3 SEQUENCE) được em áp dụng nhất quán cho toàn bộ 24 quiz (240 câu hỏi) trong dữ liệu mẫu. Việc tổng quát hóa theo `questionType` giúp em tái sử dụng một bộ component hiển thị (`VocabularyStep/ContentStep/SequenceStep`) cho cả giao diện làm bài và giao diện xem lại kết quả, đồng thời logic kiểm tra tập câu hỏi không cần phân nhánh theo loại câu hỏi.

5.2 Phân quyền tách biệt giữa học viên và quản trị viên (Hướng A)

Vấn đề. Hệ thống có hai vai trò người dùng với tập chức năng gần như không giao nhau: học viên (xem nội dung, làm quiz, theo dõi tiến độ) và quản trị viên (CRUD nội dung, quản lý câu hỏi). Trong quá trình thiết kế, em cân nhắc một hướng khác là "admin = học viên có thêm quyền" (gọi là Hướng B), nhưng nhận thấy hướng này sẽ khiến mọi trang và API phía học viên phải kiểm tra thêm điều kiện rẽ nhánh theo vai trò, làm tăng độ phức tạp và nguy cơ rò rỉ chức năng quản trị sang giao diện học viên hoặc ngược lại.

Giải pháp. Em lựa chọn Hướng A – tách biệt hoàn toàn hai luồng ADMIN và STUDENT, thực thi ở hai tầng độc lập:

- **Tầng backend (`SecurityConfig`):** em định nghĩa các nhóm endpoint theo tiền tố đường dẫn – toàn bộ `/api/v1/admin/**` yêu cầu vai trò ADMIN; các endpoint enrollment, tiến độ học, lịch sử làm quiz và `/api/v1/users/me/**` yêu cầu vai trò STUDENT. Các endpoint truy cập nội dung bài học/quiz được em bổ sung kiểm tra điều kiện đăng ký khóa học (enrollment) đối với STUDENT, trong khi ADMIN được bỏ qua điều kiện này để có thể xem trước nội dung khi chỉnh sửa.
- **Tầng frontend:** em xây dựng hai route guard riêng biệt – `ProtectedRoute` bảo vệ các route học viên (kiểm tra đã đăng nhập và vai trò STUDENT; nếu là ADMIN thì chuyển hướng sang `/admin/khoa-hoc`), và `AdminRoute` bảo vệ các route `/admin/*` (chặn STUDENT, chuyển hướng về `/dashboard`). Sau khi đăng nhập, hướng điều hướng mặc định cũng được em phân theo vai trò: ADMIN → `/admin/khoa-hoc`, STUDENT → `/dashboard`.

Kết quả. Em xây dựng hai luồng giao diện (Layout cho học viên và Admin-Layout cho quản trị viên) hoàn toàn độc lập về điều hướng, không chia sẻ thanh điều hướng (Navbar), giúp loại bỏ các điều kiện rẽ nhánh theo vai trò trong từng component. Khi kiểm thử thử công, em xác nhận: tài khoản STUDENT không thể truy cập bất kỳ route hay API nào dưới `/admin/**`; tài khoản ADMIN không bị áp dụng ràng buộc enrollment khi truy cập nội dung bài học.

5.3 Cơ chế xác thực JWT với refresh token xoay vòng

Vấn đề. Mục 2.4 đặt ra yêu cầu bảo mật: hệ thống cần xác thực người dùng qua API mà không lưu trạng thái phiên phía server, đồng thời hạn chế thiệt hại nếu access token bị lộ. Nếu sử dụng một token có thời hạn dài cho mọi request, kẻ tấn công đánh cắp được token sẽ có quyền truy cập trong suốt thời gian dài; ngược lại, nếu dùng token có thời hạn ngắn nhưng bắt người dùng đăng nhập lại liên tục thì trải nghiệm sử dụng kém.

Giải pháp. Em áp dụng mô hình hai loại token: *access token* (JWT, thời hạn 15 phút) được gửi trong header `Authorization: Bearer <token>` cho mỗi request, và *refresh token* (thời hạn 7 ngày) được lưu trong cookie `httpOnly` – không thể truy cập từ JavaScript phía client, hạn chế nguy cơ XSS đánh cắp token. Khi access token hết hạn, frontend (thông qua interceptor em viết trong `lib/axios.ts`) tự động gọi `POST /api/auth/refresh`; backend xác thực refresh token, thu hồi refresh token cũ và phát hành một cặp token mới. Nếu một refresh token đã bị thu hồi nhưng vẫn được sử dụng (dấu hiệu token bị đánh cắp và dùng lại), backend từ chối và yêu cầu đăng nhập lại. Em cũng cấu hình khóa bí mật dùng để ký JWT (`jwt.secret`) bắt buộc phải được khai báo qua biến môi trường – nếu thiếu, ứng dụng sẽ không khởi động được (`IllegalStateException`) thay vì sử dụng giá trị mặc định không an toàn.

Kết quả. Người dùng duy trì phiên đăng nhập liên tục tới 7 ngày mà không cần nhập lại mật khẩu, trong khi access token chỉ có giá trị sử dụng trong 15 phút. Cơ chế `useBootstrapAuth` em viết ở frontend tự động làm mới access token khi tải lại trang, dựa trên refresh token cookie, mang lại trải nghiệm "luôn đăng nhập" mà không phải lưu access token ở nơi có thể bị truy cập bởi script độc hại.

5.4 Tối ưu truy vấn N+1 trong thống kê khóa học theo cấp độ

Vấn đề. Trang danh sách cấp độ học (Level) cần hiển thị số lượng khóa học thuộc mỗi cấp độ (N5–N1). Cách triển khai trực tiếp – với mỗi Level, gọi một truy vấn `COUNT(*)` riêng tới bảng `courses` – dẫn tới hiện tượng *N+1 query*: 1 truy vấn lấy danh sách Level, cộng thêm N truy vấn đếm (N = số cấp độ). Với 5 cấp độ, mỗi lần tải trang sẽ phát sinh 6 truy vấn, và số lượng truy vấn tăng tuyến

tính nếu hệ thống mở rộng thêm cấp độ.

Giải pháp. Em tối ưu `LevelService` bằng cách thay thế `N` truy vấn đếm riêng lẻ bằng một truy vấn duy nhất sử dụng `GROUP BY`: phương thức `CourseRepository.countGroupByLevel()` trả về danh sách cặp (`levelId`, số lượng khóa học) trong một lần gọi CSDL. Kết quả được em nạp vào một `Map<Long, Long>` trong bộ nhớ, sau đó `LevelService` duyệt qua danh sách `Level` và tra cứu số lượng tương ứng từ `Map` này. Em áp dụng nguyên tắc tương tự cho việc đếm số bài học đã hoàn thành theo từng khóa học khi hiển thị trang `Dashboard`.

Kết quả. Số lượng truy vấn CSDL cho việc tải trang danh sách cấp độ giảm từ $N + 1$ xuống còn 2 (1 truy vấn lấy `Level`, 1 truy vấn `GROUP BY` đếm khóa học), không phụ thuộc vào số lượng cấp độ. Đây là một ví dụ cụ thể cho việc em áp dụng nguyên tắc tối ưu truy vấn ORM đã đề cập trong yêu cầu phi chức năng về hiệu năng (Mục 2.4).

5.5 Trình phát video YouTube tùy biến với luồng tự động chuyển sang quiz

Vấn đề. Theo đặc tả use case "Xem video bài học" (Mục 2.3.3), hệ thống yêu cầu: lần đầu xem một bài học, học viên phải xem hết video (tới sự kiện kết thúc) thì mới được chuyển sang trang quiz; nếu xem lại sau khi đã hoàn thành, học viên được xem tự do mà không bị ép chuyển trang. Một thẻ `<iframe>` YouTube nhúng thông thường không phát ra sự kiện trạng thái phát video, nên không thể phát hiện thời điểm video kết thúc – giải pháp thay thế phổ biến là yêu cầu người dùng tự bấm nút "Đã xem xong", vốn không đảm bảo người dùng thực sự đã xem hết.

Giải pháp. Em xây dựng component `YouTubeCustomPlayer` sử dụng `YouTube IFrame Player API` (Mục 3.6) thay cho `iframe` thô: video được nhúng với `controls:0` (ẩn control mặc định của YouTube) và em phủ lên trên một lớp giao diện điều khiển tùy biến cho play/pause, tua, tắt tiếng, và toàn màn hình. API cung cấp sự kiện `onStateChange`, em lắng nghe để xử lý hai trường hợp:

- Trạng thái `ENDED`: nếu đây là lần xem đầu tiên (em xác định qua biến `wasCompletedRef` lưu trạng thái hoàn thành *trước khi* video bắt đầu phát), frontend gửi yêu cầu cập nhật tiến độ kèm cờ hoàn thành; backend lưu tiến độ 100%, đánh dấu bài học hoàn thành, cập nhật streak, và frontend tự động điều hướng (`navigate`) sang trang quiz (`/bai-hoc/:id/quiz`); nếu bài học đã hoàn thành từ trước, hệ thống chỉ lưu tiến độ mà không điều hướng, cho phép xem lại tự do.
- Tiến độ xem: vị trí phát hiện tại được em lưu định kỳ (`debounce`, chỉ lưu khi độ lệch ≥ 5 giây so với lần lưu trước) thông qua `useProgressMutation`,

cho phép tiếp tục xem từ vị trí đã dừng ở lần truy cập sau.

Ngoài ra, khi quản trị viên nhập URL video YouTube ở trang quản lý bài học, em dùng IFrame API ở chế độ ẩn để tự động đọc thời lượng video (`getDuration()`) và điền vào trường `duration`, có giới hạn thời gian polling tối đa 10 giây để tránh treo `setInterval` nếu API không phản hồi.

Kết quả. Toàn bộ luồng "xem video → tự động chuyển quiz (lần đầu) / xem lại tự do (đã hoàn thành)" hoạt động không cần bất kỳ thao tác xác nhận thủ công nào từ người dùng, đáp ứng đúng yêu cầu UX em đặt ra. MP4Player (giải pháp cũ dùng thẻ `<video>` trực tiếp) đã được em loại bỏ hoàn toàn khỏi mã nguồn sau khi chuyển đổi sang YouTube IFrame API.

5.6 Tích hợp thanh toán VNPay với cơ chế đăng ký khóa học tự động

Vấn đề. Mục 2.3.2 (đặc tả use case "Đăng ký khóa học") yêu cầu hệ thống hỗ trợ cả khóa học miễn phí và khóa học có phí. Với khóa học có phí, hệ thống cần đảm bảo người dùng chỉ được ghi nhận đăng ký sau khi giao dịch thanh toán đã được xác thực là hợp lệ – nếu chỉ dựa vào việc frontend gọi API "đăng ký" sau khi chuyển hướng về từ cổng thanh toán, một người dùng có thể giả mạo việc gọi API này mà không thực sự thanh toán. Bên cạnh đó, do callback từ VNPay là một request GET không yêu cầu xác thực người dùng (`permitAll()`), em cũng cần lường trước khả năng request này bị gọi lại nhiều lần cho cùng một giao dịch.

Giải pháp. Như mô tả trong biểu đồ trình tự ở Mục 4.2.2 (Hình 4.5), em thiết kế việc tạo bản ghi `UserCourseEnrollment` cho khóa học có phí không được thực hiện trực tiếp từ một API gọi bởi frontend, mà chỉ được thực hiện bên trong `PaymentService` – duy nhất khi `VnpayService` xác thực thành công chữ ký HMAC-SHA512 trong request callback (`GET/api/v1/payments/vnpay-return`) gửi từ máy chủ VNPay. Quy trình cụ thể:

1. Frontend gọi `POST/api/v1/payments/create`; backend tạo bản ghi `Payment` ở trạng thái chờ (`PENDING`) và trả về URL thanh toán đã được ký. Nếu người dùng đã có một giao dịch `PENDING` chưa hoàn tất cho cùng khóa học (ví dụ do bấm mua nhiều lần), em tái sử dụng mã giao dịch (`vnpTxnRef`) của giao dịch đó để sinh lại URL thay vì tạo thêm một bản ghi `Payment` mới, tránh tích lũy giao dịch chờ không cần thiết.
2. Người dùng hoàn tất (hoặc hủy) thanh toán trên giao diện VNPay sandbox.
3. VNPay gọi lại `vnpay-return` kèm các tham số giao dịch và chữ ký; `VnpayService` tính lại chữ ký từ các tham số nhận được bằng *hash secret* và so sánh với chữ ký nhận được.

4. Nếu khớp và giao dịch thành công, `PaymentService` cập nhật `Payment` sang trạng thái thành công và gọi `EnrollmentService.enroll(...)`; nếu không khớp hoặc giao dịch thất bại, `Payment` được đánh dấu thất bại và không có enrollment nào được tạo. Nếu `Payment` đã ở trạng thái thành công từ một lần gọi callback trước đó, em chặn xử lý lại ngay từ đầu và trả về kết quả thành công mà không ghi đè thông tin giao dịch gốc hay gọi enroll thêm lần nữa.
5. Backend chuyển hướng (redirect) trình duyệt về `/thanh-toan/ket-qua` của frontend kèm trạng thái, để `PaymentResultPage` hiển thị thông báo phù hợp và nút "Vào học ngay" (nếu thành công).

Đối với khóa học miễn phí (`price = 0`), luồng VNPay được bỏ qua hoàn toàn – `EnrollmentService.enroll(...)` được gọi trực tiếp từ API đăng ký khi người dùng bấm "Đăng ký học".

Kết quả. Việc xác thực enrollment hoàn toàn dựa trên kết quả xác thực phía server (chữ ký HMAC-SHA512 từ VNPay), loại bỏ khả năng giả mạo trạng thái thanh toán từ phía client; cơ chế chặn xử lý lại giao dịch đã thành công giúp endpoint callback an toàn trước việc bị gọi lại nhiều lần. Các trường hợp kiểm thử TC-Pay-01 đến TC-Pay-05 (Mục 4.4) đã xác nhận đầy đủ các nhánh: đăng ký miễn phí, thanh toán thành công, thanh toán thất bại/giả mạo, và gọi lại callback trùng lặp đều được xử lý đúng.

CHƯƠNG 6. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

6.1 Kết luận

6.1.1 So sánh với các nền tảng tương tự

So với các nền tảng em đã khảo sát ở Mục 2.1 (Bảng 2.1), NihongoFlow đáp ứng đồng thời các tiêu chí mà từng nền tảng riêng lẻ chưa đáp ứng đủ: hệ thống có nội dung video bài giảng được tổ chức theo cấp độ JLPT (như Duolingo, JapanesePod101), gắn kèm bài kiểm tra kiến thức ngay sau mỗi video (khắc phục hạn chế của các kênh YouTube thuần túy), hỗ trợ đa dạng nhóm câu hỏi (từ vựng, nội dung, trình tự – vượt qua hình thức câu hỏi đơn điệu của WaniKani), và toàn bộ giao diện được em bản địa hóa hoàn toàn bằng tiếng Việt cho người học Việt Nam. Ngoài ra, em bổ sung các yếu tố tạo động lực học tập (streak theo ngày, lịch sử và xem lại kết quả làm bài) cũng như cơ chế đăng ký khóa học có/không thu phí – những yếu tố ít xuất hiện đồng thời trong các nền tảng em đã khảo sát.

6.1.2 Kết quả đạt được

Qua quá trình thực hiện, em đã hoàn thành các mục tiêu đề ra ở Mục 1.2:

- Xây dựng hoàn chỉnh backend RESTful API bằng Spring Boot, với xác thực JWT (access token + refresh token xoay vòng), phân quyền RBAC giữa STUDENT và ADMIN, và quản lý lược đồ CSDL MySQL qua Flyway.
- Xây dựng nội dung học theo cấu trúc phân cấp Cấp độ – Khóa học – Bài học – Quiz, với dữ liệu mẫu gồm 5 cấp độ, 10 khóa học, 24 bài học và 240 câu hỏi quiz chia theo 3 nhóm (VOCABULARY/CONTENT/SEQUENCE).
- Xây dựng trình phát video YouTube tùy biến với theo dõi tiến độ và luồng tự động chuyển sang quiz khi xem hết video lần đầu, cho phép xem lại tự do sau khi hoàn thành.
- Xây dựng hệ thống quiz với cách chấm điểm riêng cho từng loại câu hỏi, lưu lịch sử làm bài và cho phép xem lại đáp án đúng/sai theo từng câu.
- Xây dựng cơ chế theo dõi streak học tập theo ngày (tính theo lịch, không phụ thuộc thời gian thực của client).
- Xây dựng cơ chế đăng ký khóa học, hỗ trợ cả khóa học miễn phí (đăng ký trực tiếp) và khóa học có phí (qua cổng thanh toán VNPay sandbox với xác thực chữ ký HMAC-SHA512 và chống xử lý lại callback trùng lặp).
- Xây dựng khu vực quản trị riêng biệt (tách hoàn toàn khỏi giao diện học viên) cho phép quản lý CRUD khóa học, bài học và chỉnh sửa nội dung câu hỏi quiz.

- Xây dựng frontend bằng React + TypeScript (strict mode), sử dụng TanStack Query và Zustand để quản lý dữ liệu và trạng thái, theo phong cách thiết kế tối giản, nền tối, nhất quán trên toàn hệ thống, kèm bộ kiểm thử tự động cho các logic quan trọng ở cả hai phía backend và frontend.

Sáu đóng góp kỹ thuật em tâm đắc nhất – thiết kế hệ thống quiz mở rộng theo kiểu câu hỏi, phân quyền tách biệt ADMIN/STUDENT, cơ chế JWT refresh token xoay vòng, tối ưu truy vấn N+1, trình phát video YouTube tùy biến, và tích hợp thanh toán VNPAY – đã được em trình bày chi tiết tại Chương 5.

6.1.3 Hạn chế còn tồn tại

Bên cạnh các kết quả đạt được, em nhận thấy hệ thống còn một số hạn chế:

- **Chưa hỗ trợ kéo-thả sắp xếp thứ tự bài học:** thứ tự bài học trong khóa học hiện được lưu qua trường `order` nhưng em chưa xây dựng giao diện kéo-thả để quản trị viên thay đổi thứ tự trực quan.
- **Chưa phân trang lịch sử làm quiz:** trang Dashboard hiển thị toàn bộ lịch sử các lượt làm quiz của học viên mà em chưa áp dụng phân trang, có thể ảnh hưởng hiệu năng khi số lượng lượt làm bài lớn.
- **Giao diện chưa tối ưu cho thiết bị di động:** em thiết kế hệ thống theo hướng desktop-first; trải nghiệm trên màn hình nhỏ chưa được em kiểm thử và tối ưu đầy đủ.
- **Dạng câu hỏi quiz còn giới hạn ở ba nhóm hiện có:** mặc dù thiết kế đã tính tới khả năng mở rộng (Mục 5.1), các dạng câu hỏi nâng cao hơn (nghe-chọn đáp án có audio riêng, nối cặp từ vựng) chưa được em triển khai trong phạm vi đồ án này.

6.1.4 Bài học kinh nghiệm

Quá trình thực hiện đồ án độc lập mang lại cho em một số bài học kinh nghiệm quan trọng. Thứ nhất, việc đầu tư thời gian thiết kế schema CSDL tổng quát ngay từ đầu (đặc biệt là cách tổ chức bảng `questions`) giúp em tránh được việc phải refactor lớn khi yêu cầu về loại câu hỏi thay đổi trong quá trình phát triển – em chỉ cần bổ sung thêm cột cho schema hiện có thay vì thiết kế lại toàn bộ luồng chấm điểm. Thứ hai, việc phân tách rõ ràng giữa hai vai trò người dùng ngay từ tầng kiến trúc (cả backend lẫn frontend) giúp em giảm đáng kể số lượng lỗi liên quan tới phân quyền so với cách kiểm tra vai trò rải rác trong từng chức năng. Thứ ba, với một đồ án do một mình em phát triển toàn bộ (cả backend lẫn frontend), việc lựa chọn các công nghệ có tài liệu phong phú và cộng đồng hỗ trợ tốt (Spring Boot, React, TanStack Query) giúp em tiết kiệm đáng kể thời gian xử lý các vấn đề kỹ

thuật phát sinh, từ đó dành nhiều thời gian hơn cho việc thiết kế nghiệp vụ và trải nghiệm người dùng.

6.2 Hướng phát triển

Trên cơ sở các hạn chế đã nêu, em đề xuất các hướng phát triển tiếp theo, chia thành hai nhóm.

Hoàn thiện các chức năng hiện có:

- Bổ sung giao diện kéo-thả (sử dụng thư viện `@dnd-kit/sortable`) để quản trị viên sắp xếp lại thứ tự bài học, gọi API `PATCH/admin/lessons/:id/order` đã được em dự kiến trong thiết kế.
- Bổ sung phân trang cho danh sách lịch sử làm quiz trên trang Dashboard, đồng thời áp dụng cho các danh sách dài khác nếu phát sinh (ví dụ danh sách người dùng phía quản trị).
- Tối ưu giao diện cho thiết bị di động: điều chỉnh layout, kích thước chữ và vùng chạm cho các trang chính (Dashboard, trang xem video, trang quiz).

Mở rộng tính năng mới:

- Mở rộng hệ thống quiz với các dạng câu hỏi mới tận dụng thiết kế tổng quát đã có (Mục 5.1): câu hỏi nghe kèm audio, câu hỏi nối cặp từ vựng – chỉ cần bổ sung giá trị `questionType` mới và component hiển thị tương ứng ở frontend.
- Xây dựng hệ thống gợi ý lộ trình học cá nhân hóa dựa trên lịch sử làm quiz và tốc độ học của từng người dùng, có thể bắt đầu từ các quy tắc thống kê đơn giản trước khi cân nhắc áp dụng các mô hình học máy.
- Bổ sung thông báo nhắc nhở học tập (ví dụ qua email) khi người dùng có nguy cơ mất chuỗi streak, nhằm tăng tỷ lệ duy trì học tập.
- Mở rộng phương thức thanh toán ngoài VNPay (ví dụ Momo, ZaloPay) để tăng tính tiện lợi cho người dùng tại Việt Nam.

TÀI LIỆU THAM KHẢO

- [1] The Japan Foundation, *Survey report on japanese-language education abroad 2021*, 2021. Accessed: Feb. 1, 2026. [Online]. Available: <https://www.jpjf.go.jp/e/project/japanese/survey/result/survey21.html>
- [2] Japan External Trade Organization (JETRO), *Survey on business conditions of japanese-affiliated companies in asia and oceania*, 2023. Accessed: Feb. 1, 2026. [Online]. Available: <https://www.jetro.go.jp/en/news/survey/>
- [3] H. L. Roediger and J. D. Karpicke, “Test-enhanced learning: Taking memory tests improves long-term retention,” *Psychological Science*, vol. 17, no. 3, pp. 249–255, 2006.
- [4] R. T. Fielding, *Architectural styles and the design of network-based software architectures*, PhD dissertation, University of California, Irvine, 2000. Accessed: Feb. 1, 2026. [Online]. Available: <https://www.ics.uci.edu/~fielding/pubs/dissertation/top.htm>
- [5] VMware (Spring Team), *Spring boot reference documentation*, 2024. Accessed: Feb. 1, 2026. [Online]. Available: <https://docs.spring.io/spring-boot/docs/current/reference/html/>
- [6] M. B. Jones, J. Bradley, and N. Sakimura, *RFC 7519: JSON web token (JWT)*, Internet Engineering Task Force (IETF), 2015. Accessed: Feb. 1, 2026. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc7519>
- [7] Oracle Corporation, *Mysql 8.0 reference manual*, 2024. Accessed: Feb. 1, 2026. [Online]. Available: <https://dev.mysql.com/doc/refman/8.0/en/>
- [8] Meta Platforms, Inc., *React documentation*, 2024. Accessed: Feb. 1, 2026. [Online]. Available: <https://react.dev/>
- [9] Tailwind Labs Inc., *Tailwind css documentation*, 2024. Accessed: Feb. 1, 2026. [Online]. Available: <https://tailwindcss.com/docs>
- [10] Google LLC, *YouTube IFrame player API reference*, 2024. Accessed: Feb. 1, 2026. [Online]. Available: https://developers.google.com/youtube/iframe_api_reference
- [11] VNPAY, *Tài liệu tích hợp cổng thanh toán VNPAY*, 2024. Accessed: Feb. 1, 2026. [Online]. Available: <https://sandbox.vnpayment.vn/apis/docs/>

PHỤ LỤC

A. ĐẶC TẢ USE CASE BỔ SUNG

Phụ lục này em đặc tả thêm hai use case không được trình bày chi tiết trong Chương 2 do giới hạn dung lượng chương, nhưng vẫn là các chức năng quan trọng của hệ thống NihongoFlow.

A.1 Đặc tả use case “Xem tổng quan tiến độ và streak học tập”

Tác nhân: Học viên.

Mô tả: Học viên xem trang Dashboard, hiển thị tiến độ hoàn thành của từng khóa học đã đăng ký, chuỗi ngày học liên tục hiện tại và chuỗi dài nhất từng đạt được, cùng lịch sử các lượt làm quiz gần đây.

Tiền điều kiện: Học viên đã đăng nhập.

Hậu điều kiện: Không thay đổi dữ liệu hệ thống (use case chỉ đọc).

Luồng sự kiện chính:

1. Học viên truy cập trang /dashboard.
2. Hệ thống gọi đồng thời các API: danh sách khóa học đã đăng ký kèm phần trăm hoàn thành (`completedLessons / totalLessons`), thông tin streak hiện tại và streak dài nhất, và danh sách các lượt làm quiz gần đây nhất.
3. Hệ thống hiển thị: thẻ thống kê streak (hiện tại/dài nhất), danh sách khóa học đang học kèm thanh tiến độ, và bảng lịch sử làm quiz (tên bài học, điểm số, ngày làm bài, trạng thái đạt/không đạt).
4. Học viên nhấp vào một lượt làm quiz trong lịch sử.
5. Hệ thống gọi `GET/api/v1/users/me/quiz-attempts/:id` và mở `AttemptReviewModal`, hiển thị toàn bộ 10 câu hỏi của lượt làm bài đó, nhóm theo loại câu hỏi (VOCABULARY/CONTENT/SEQUENCE), tô màu xanh cho đáp án đúng và màu đỏ cho đáp án học viên đã chọn sai.

Luồng rẽ nhánh:

- Nếu học viên chưa đăng ký khóa học nào, danh sách khóa học hiển thị trạng thái rỗng kèm thông báo gợi ý khám phá khóa học (không hiển thị khoảng trống).
- Nếu chuỗi ngày học bị gián đoạn quá 24 giờ kể từ `lastActivityDate`, `currentStreak` được tính lại bằng 0 ở phía server trước khi trả về cho frontend.

A.2 Đặc tả use case “Ẩn/hiện khóa học” (Quản trị viên)

Tác nhân: Quản trị viên.

Mô tả: Quản trị viên có thể ẩn một khóa học khỏi danh sách khóa học hiển thị cho học viên (ví dụ khi nội dung chưa hoàn thiện), hoặc hiện lại một khóa học đã ẩn trước đó, mà không cần xóa dữ liệu khóa học.

Tiền điều kiện: Người dùng đã đăng nhập với vai trò ADMIN; khóa học cần thao tác đã tồn tại.

Hậu điều kiện: Trường `Course.hidden` của khóa học được đảo trạng thái (`true` $\leftrightarrow$ `false`); nếu `hidden = true`, khóa học không xuất hiện trong danh sách khóa học và không thể truy cập trực tiếp bởi học viên (kể cả khi biết đường dẫn).

Luồng sự kiện chính:

1. Quản trị viên truy cập trang `/admin/khoa-hoc`, xem bảng danh sách khóa học kèm cột trạng thái hiển thị.
2. Quản trị viên nhấp nút ẩn/hiện tương ứng với một khóa học.
3. Frontend gọi `PATCH/api/v1/admin/courses/:id` với trường `hidden` đảo ngược giá trị hiện tại.
4. Backend (`AdminCourseService`) cập nhật bản ghi `Course` và trả về DTO đã cập nhật.
5. Frontend cập nhật lại danh sách khóa học (`invalidate cache TanStack Query`) và hiển thị trạng thái mới trong bảng.

Luồng rẽ nhánh:

- Nếu một học viên đã đăng ký khóa học trước khi khóa học bị ẩn, học viên đó vẫn được tiếp tục truy cập các bài học đã đăng ký – cờ `hidden` chỉ ảnh hưởng tới việc khóa học có xuất hiện trong danh sách/khám phá khóa học mới hay không, được em kiểm tra ở `CourseService.getCourse` và `getLessons` đối với vai trò STUDENT.
- Quản trị viên (vai trò ADMIN) luôn nhìn thấy và truy cập được khóa học bất kể trạng thái `hidden`.